



Design and Analysis of Algorithms

Block-1 Unit-1

Basics of an Algorithm and Its Properties

Topics to be Covered

Introduction

Example of an Algorithm

Basics Building Blocks of Algorithms

A Survey of Common Running Time

Analysis & Complexity of Algorithm

Types of Problems

Topics to be Covered

Problem Solving Techniques

Deterministic and Stochastic Algorithms

Important Topics

Summary

Introduction

- Studying algorithms is an exciting subject in computer science discipline.
- We come across a large number of interesting problems and techniques to solve to solve these problems.
- Not every problem can be solved with the existing techniques but majority of them can be. But let us define what is an algorithm first.
- The word algorithm is derived from the mathematician of the ninth century Abdullah Jafar Muhammad ibn Musa Al-khowarizmi.
- The word ‘alkhowarizmi’ is ‘Algorismus’ in Latin which became Algorithm after his name.

Example of an Algorithm

- An algorithm is not a coding instruction rather it is a sequence of tasks written in common language, if executed produces certain output within a time frame.
- An algorithm is completely independent of programming language.

Example of an Algorithm

- For a good algorithm, must satisfy the following characteristics
 - **Input:** There must be a finite number of inputs for the algorithm.
 - **Output:** There must be some output produced as a result of execution of the algorithm.
 - **Definiteness:** There must be a definite sequence of operations for transformation of input into output.
 - **Effectiveness:** Every step of the algorithm should be basic and essential.
 - **Finiteness:** The transformation of input to output must be achieved in finite steps.

Example of an Algorithm

- Following are desirable characteristics of an algorithm:
 - The algorithm should be general and is able to solve several cases.
 - The algorithms should use resources efficiently, i.e. takes less time and memory in producing the result.
 - The algorithms should be understandable so that anyone can understand and apply it to own problem.
 - The algorithm should follow the uniqueness such that each instruction of the algorithm is unambiguous and clear.

Example of an Algorithm

- Let us find the GCD of $a = 1071$ and $b = 462$ using Euclid's algorithm :-
 - Divide $a=1071$ by $b=462$ and store the remainder in r . $r = 1071 \% 462$ (here, $\%$ represents the remainder operator). $R = 147$
 - If $r = 0$, the algorithm terminates and b is the GCD. Otherwise, go to Step 3. Here, r is not zero, so we will go to Step 3.
 - The integer will get the current value of integer b and the new value of integer b will be the current value of r . Here, $a=462$ and $b=147$
 - Go back to Step 1.

Basics Building Blocks of Algorithms

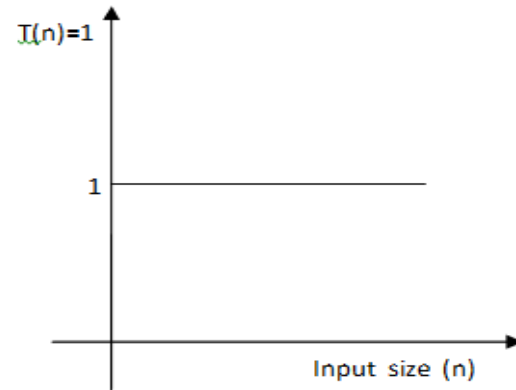
- An algorithm is a procedural way to write the solution of a problem. It is designed with five basic building blocks, namely
 - Sequencing/Step by step actions
 - Selection/Decision
 - Iteration/Repetition or Loop
 - Procedure
 - Recursion

A Survey of Common Running Time

- To compare two algorithms for a problem, running time is generally used which is defined as the time taken by an algorithm in generating the output.
- An algorithm is better if it takes less running time. The “time” here is not necessarily the clock time. However, this measure should be invariant to any hardware used.
- The running time of an algorithm can be represented in terms of the number of operations executed for a given input. More the number of operations, the larger the running time of an algorithm.
- This running time of an algorithm for producing the output is also known as time complexity.

A Survey of Common Running Time

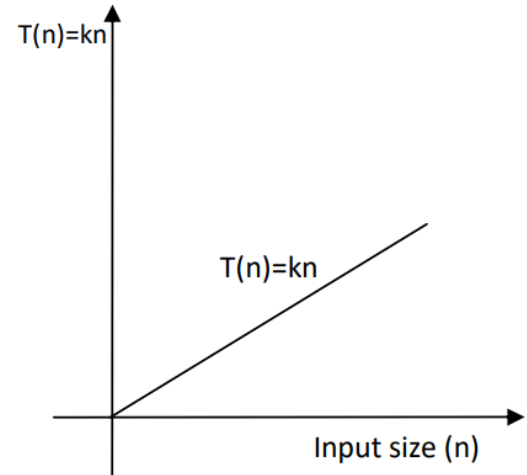
- Following are the generalized form of running time for the algorithms:
 - **Constant Time($O(1)$):** If the running time does not depend on the input size (n) then it is known as constant running time. It can be represented as



A Survey of Common Running Time

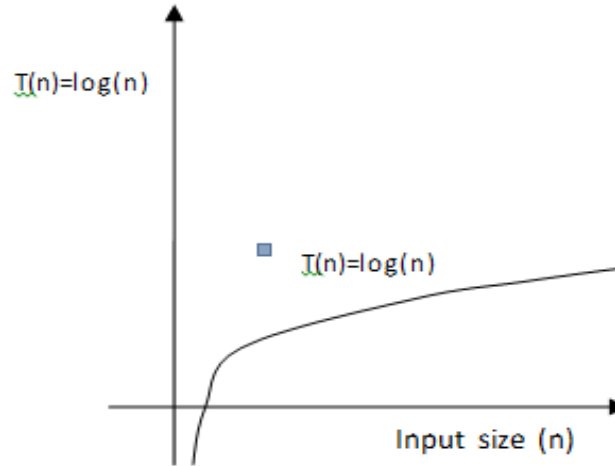
- **Linear Time $O(kn)$:** If the time complexity is at most a constant factor times the size of the input, then it is known as linear time complexity and is presented as $T(n) \leq k \cdot n$ where k is a constant or $T(n) = O(n)$.

```
minimum = a[1]
for i = 2 to n
  if a[i] < minimum
    minimum = a[i]
  end
end if
```



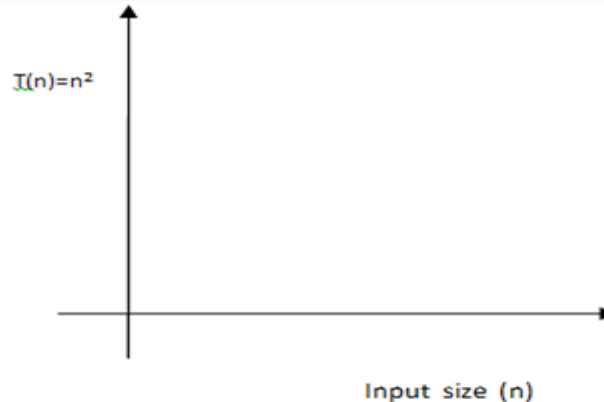
A Survey of Common Running Time

- **Logarithmic Time($\log(n)$):** The time complexity of an algorithm is proportional to the logarithm of the input size and it is known as logarithmic time complexity and depicted as $O(\log n)$ time.



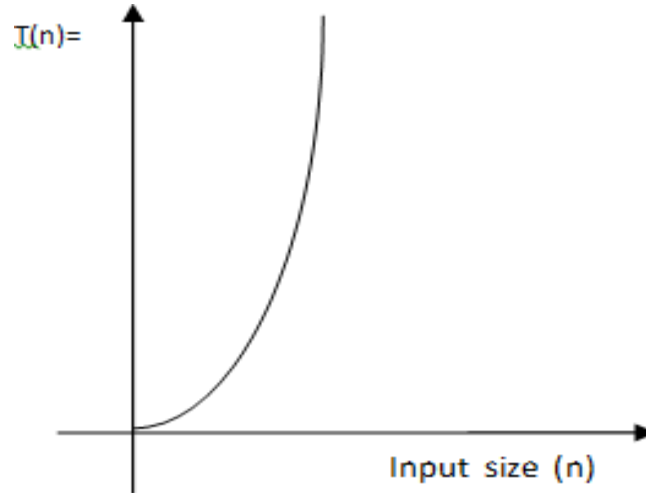
A Survey of Common Running Time

- **Quadratic Time:** ($T(n) = O(n^2)$)- The algorithm is a pair of nested loops. The outer loop iterates $O(n)$ time and for each iteration the inner loop takes $O(n)$ time so we get $O(n^2)$ by multiplying these two factors of n , is useful for problem for small input size or elementary sorting algorithms.



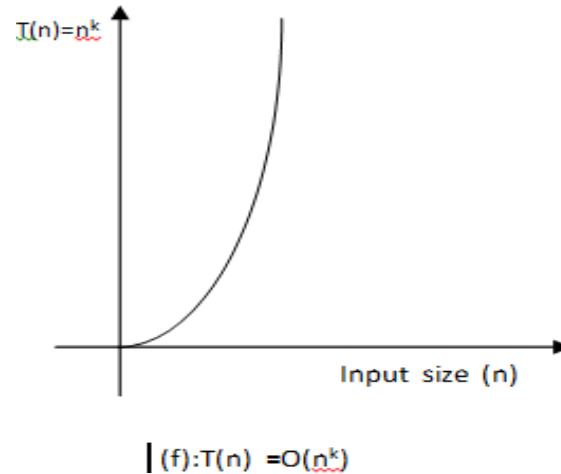
A Survey of Common Running Time

- **Cubic Time:** ($T(n) = O(n^3)$): It often occurs when the algorithm is having three nested loops and each loop has a maximum n iterations. .



A Survey of Common Running Time

- **Polynomial Time**($O(n^k)$):-This running time is obtained when the search over all subsets of a set of a size k is performed.



A Survey of Common Running Time

Exponential Time:

Beyond the polynomial time complexity there are other two types of bounds :

- Exponential Time $O(2^n)$
- Factorial Time $O(n!)$

Analysis & Complexity of Algorithm

- The term "analysis of algorithms" means to understand the complexity of an algorithm in terms of time complexity and storage requirement.
- System performance is directly dependent on the efficiency of algorithm in terms of both the time complexity as well the memory.
- There are 3 cases, to find the complexity function $f(n)$:
 - **Worst-case** – The maximum number of steps taken on any instance of size.
 - **Best-case** – The minimum number of steps taken on any instance of size.
 - **Average case** – The number of steps taken on average for all instances.

Analysis & Complexity of Algorithm

- To understand the Best, Worst and Average cases of an algorithm, consider a linear array

Algorithm: (**Linear search**)

/* **Input:** A linear list A with n elements and a searching element .

Output: Finds the location LOC of x in the array A (by returning an index) or return LOC=0 to indicate x is not present in A.*/

1. [Initialize]: Set $K=1$ and $LOC=0$.
2. Repeat step 3 and 4 while $(LOC == 0 \ \&\& \ K \leq n)$
3. If $(x == A[K])$
4. {
5. $LOC=K$ 6. $K=K+1$; 7. }
8. If $(LOC == 0)$
9. Print (" x is not present in the given array A);
10. Else
11. Print f(" x is present in the given array A at location A [LOC]);
12. Exit [end of algorithm]

Analysis & Complexity of Algorithm

- **Analysis of linear search algorithm**

- The complexity of the search algorithm is given by the number C of comparisons between x and array elements $A[K]$.
- **Best case:** Clearly the best case occurs when x is the first element in the array A . That is . In this case $C(n) = 1$
- **Worst case:** Clearly the worst case occurs when x is the last element in the array A or x is not present in given array A . In this case, we have

$$C(n) = n$$

Analysis & Complexity of Algorithm

- **Average case:** The searched element x appear in array A , and it is equally likely to occur at any position in the array the number of comparisons can be any of numbers $1, 2, 3, \dots, n$, and each number occurs with the probability $p=1/n$ then

$$\begin{aligned}C(n) &= 1 \cdot \frac{1}{n} + 2 \cdot \frac{1}{n} + \dots + n \cdot \frac{1}{n} \\&= (1 + 2 + \dots + n) \cdot \frac{1}{n} \\&= n(n+1) \cdot \frac{1}{n} = \frac{n+1}{2}\end{aligned}$$

Types of Problems

Sorting

Searching

Graph problems

Combinatorial problems

Geometric problems

Numerical problems

Types of Problems

Sorting

- The sorting is the process to arrange the given set of items in a certain
- order, assuming that the nature of the items allow such an ordering. For example, sorting a set of numbers in increasing or decreasing order and sorting the character strings, like names, in an alphabetical order. For any sorting algorithm following two characteristics are desirable:
 - Stability
 - In-place

Types of Problems

Searching

- Searching is finding an element, referred as search key, in a given set of items. Searching is one of the most important and frequently performed operation on any dataset/database.

Types of Problems

Graph Problems

- It is helpful for researchers to map a computational problem to a graph problem. Many computational problems can be solved using graph.
- Most of the problems like: visiting all the nodes of a graph, routing in networks, finding the minimum cost path, i.e.
- The shortest path, path with minimum delay etc. Can be solved efficiently with graph algorithms.

Types of Problems

Combinatorial Problems

- These types of problems have a combination of solutions i.e. more than one solution are possible. The aim of the combinatorial problems is to find permutations, combinations, or subsets, satisfying the given conditions.

Types of Problems

Geometric Problems

- These types of problems have a combination of solutions i.e. more than one solution are possible.
- The aim of the combinatorial problems is to find permutations, combinations, or subsets, satisfying the given conditions.

Types of Problems

- Following are widely known classic problems of computational geometry:
 - The closest-pair problem
 - The convex-hull problem
- The closest-pair problem is to find the closest pair out of a given set of points in the plane.
- In the convex-hull problem, the smallest convex polygon is to be constructed so that it includes all the points of a given set.

Types of Problems

- **Numerical Problems**

- Problems of numerical computing nature are simultaneous linear equations ,differential equations, definite integration, and statistics.
Most of the numerical problems could be solved approximately.
- The biggest drawback of numerical algorithms is the accumulation of errors over the multiple iterations, due to rounding off the approximated result at each iteration.

Problem Solving Techniques

Divide and Conquer Approach

- This is one of the popular approaches in which a problem is divided into smaller sub problems. These sub problems are further divided into smaller sub problems until they can no longer be divided.

Problem Solving Techniques

- An algorithm, following divide & conquer technique, involves following steps:
 - Step 1. Divide the problem (top level) into a set of sub-problems (lower level).
 - Step 2. Solve every sub-problem individually by recursive approach.
 - Step 3. Merge the solution of the sub-problems into a complete solution of the problem.

Problem Solving Techniques

- **Greedy Technique**

- Using Greedy approach, optimization problems are solved efficiently.
- In an optimization problem, the given set of input values are either to be maximized or minimized, subject to some constraints or conditions.
- Greedy algorithm always picks the best choice (greedy approach) out of many at a particular moment to optimize a given objective

Problem Solving Techniques

- The greedy method chooses the local optimum at each step and this decision may result in overall non-optimum or optimum solution.
- Following are some of the examples of the greedy approach.
 - Kruskal's Minimum Spanning Tree
 - Prim's Minimal Spanning Tree
 - Dijkstra's shortest path
 - Knapsack Problem

Problem Solving Techniques

Dynamic Programming

- Dynamic Programming approach is a bottom-up approach which involves finding solution of all sub-problems, saving these partial results, and then reusing them to solve larger sub-problems until the solution to the original problem is obtained.

Problem Solving Techniques

- **Branch and Bound**

- Branch and bound algorithm efficiently solves the discrete and combinatorial optimization problems. In branch-and-bound algorithm, a rooted tree is formed with the full solution set at the root.
- The algorithm explores the branches of this tree, representing the subsets of the solution set.

Problem Solving Techniques

- **Randomized Algorithms**

- In a randomized algorithm, a random number is selected at any stage of the solution and is used for computation of the solution, that's why it is called as randomized algorithm.
- In other words it can be said that algorithms that make random choices for faster solutions are known as randomized algorithms.

Problem Solving Techniques

- **Backtracking Algorithm**

- Backtracking algorithm is like creating checkpoints while exploring new solutions. It works analogous to depth-first search. It searches all the possible solutions.
- During the exploration of solutions, if a solution doesn't work, it back-track to the previous place and then find the other alternatives to get the solution. If there are no more choice points the search fails.

Deterministic & Stochastic algorithm

Algorithms can be categorized either deterministic or stochastic in nature. An algorithm is deterministic if the next output can be predicted/ determined from the input and the state of the program, whereas stochastic algorithms are random in nature.

Problems with unpredictable result cannot be solved using deterministic approach.

Important Topics

Building Blocks of Algorithms

Sequencing Selection and Iteration

Procedure and Recursion

Common Running Time

Analysis and Complexity of Algorithm

Problem solving Techniques

Deterministic and Stochastic Algorithm

Summary

- In this session, we have seen that, an algorithm is independent from a programming language. Algorithm is designed to understand and analyze the solution of a computational problem.
- An algorithm can be analyzed in terms of time complexity and space complexity. To evaluate algorithms of a problem, time and space complexities are considered. Algorithm, taking less time to produce the desired output.

*Thank
You*



Design and Analysis of Algorithms

Block-1 Unit-2

Asymptotic Bounds

Topics to be Covered

Introduction

Some Useful Mathematical Functions & Notations

Mathematical Expectation

Principal of Mathematical Induction

Efficiency of an Algorithm

Asymptotic Functions & Notations

Important Topics

Summary

Introduction

- In the last session, we have discussed about algorithms and its basic properties. We also discussed about deterministic and stochastic algorithms.
- In this session, we will discuss the process to compute complexities of different algorithms, useful mathematical functions and notations, principle of mathematical induction, and some well known asymptotic functions.
- Algorithmic complexity is an important area in computer science. If we know complexities of different algorithms then we can easily answer the following questions-
 - How long will an algorithm/ program run on an input ?
 - How much memory will it require ?
 - Is the problem solvable ?

Some Useful Mathematical Functions And Notations

- Functions & Notations : Just to put the subject matter in proper context, we recall the following notations and definitions.
- **Functions & Notations**
 - $N = \{1, 2, 3, \dots\}$
 - $I = \{\dots, -2, -1, 0, 1, 2, \dots\}$
 - R = set of Real numbers.
- **Notation:** If $a_1, a_2 \dots a_n$ are n real variables/numbers

Some Useful Mathematical Functions And Notations

- **Summation**

Suppose we are having a sequence of numbers 1, 2,...,n where n is a integer number, the finite sum of these sequences, i.e. 1+2+.....n can be denoted as: $\sum_{i=1}^n i$, where Σ is called sigma symbol

$\sum_{i=1}^n i$ is in arithmetic series and has the values

Sum of sequences

(i)
$$\sum_{i=1}^n i = 1 + 2 + \dots n = \frac{n(n+1)}{2}$$

(ii)
$$\sum_{i=1}^n i^2 = 1^2 + 2^2 + \dots n^2 = \frac{n(n+1)(2n+1)}{6}$$

(iii)
$$\sum_{i=1}^n i^3 = 1^3 + 2^3 + 3^3 + \dots n^3 = \frac{n^2(n+1)^2}{4}$$

Some Useful Mathematical Functions And Notations

- **Product**

The expression $1 \times 2 \times \dots \times n$

is denoted in shorthand as

$$\prod_{i=1}^n i$$

Some Useful Mathematical Functions And Notations

- **Function**

- For two given sets A and B a rule f which associates with each element of A, a unique element of B, is called a function from A to B. If f is a function from a set A to a set B then we denote the fact by $f: A \rightarrow B$. For example the function f which associates the cube of a real number with a given real number x, can be written as $f(x) = x^3$.
- Suppose the value of x is 2 there f maps 2 to 8

Some Useful Mathematical Functions And Notations

- **Floor Function:** Let x be a real number. The floor function maps each real number x to the integer, which is the greatest of all integers less than or equal to x . Example :-

$$\lfloor 3.5 \rfloor = 3, \lfloor -3.5 \rfloor = -4, \lfloor 8 \rfloor = 8.$$

- **Ceiling Function:** Let x be a real number. The ceiling function denote each real number x to the integer, which is the least of all integers greater than or equal to x . Example :-

$$\lceil 3.5 \rceil = 4, \lceil -3.5 \rceil = -2, \lceil 8 \rceil = 8$$

Some Useful Mathematical Functions And Notations

- **Multiplying by a constant or an expression**
 - If C is a constant,

$$C \sum_{k=m}^n a_k = \sum_{k=m}^n C \cdot a_k$$

Example :

$$2 \sum_{n=1}^4 \frac{1}{n^2} = \sum_{n=1}^4 \frac{2}{n^2}$$

$$2 \left(1 + \frac{1}{2^2} + \frac{1}{3^2} + \frac{1}{4^2} \right) = 2 + \frac{2}{2^2} + \frac{2}{3^2} + \frac{2}{4^2}$$

Example :

$$r \sum_{k=1}^n r^{k-1} = \sum_{k=1}^n r \cdot r^{k-1} = \sum_{k=1}^n r^k$$

$$r(r^0 + r^1 + \dots + r^{n-1}) = r \cdot r^0 + r \cdot r^1 + \dots + r \cdot r^{n-1} = r^1 + r^2 + \dots + r^n$$

Some Useful Mathematical Functions And Notations

- **Logarithms**

- Logarithms are important mathematical tools which are widely used in analysis of algorithms. For n , a some important formulas related to logarithms are given below

- $\log_a(bc)$ = $\log_a b + \log_a c$
- $\log_a(b^n)$ = $n \log_a b$
- $\log_a b$ = $\log_a b$
- $\log_a(1/b)$ = $-\log_a b$
- $\log_b a$ = $1/\log_a b$
- $a \log_b c$ = $c \log_b a$

Some Useful Mathematical Functions And Notations

- **Modular Arithmetic/Mod Function**

- The modular function or mod function returns the remainder after a number

(called dividend) is divided by another number called divisor.

- **Definition**

- **$b \bmod n$:** if n is a given positive integer and b is any integer, then $b \bmod n = r$ where $0 \leq r < n$ and $b = k * n + r$

Mathematical Expectation

- In average-case analysis of algorithms, we need the concept of Mathematical expectation. In order to understand the concept better, let us first consider an example.
- Suppose, the students of MCA, who completed all the courses in the year 2005, had the following distribution of marks.

Mathematical Expectation

Range of marks

**Percentage of students
who scored in the range**

0% to 20%	08
20% to 40%	20
40% to 60%	57
60% to 80%	09
80% to 100%	06

- If a student is picked up randomly from the set of students under consideration, what is the % of marks expected of such a student? After scanning the table given above, we intuitively expect the student to score around the 40% to 60% class, because, more than half of the students have scored marks in and around this class.

Mathematical Expectation

- Assuming that marks within a class are uniformly scored by the students in the class, the above table may be approximated by the following more concise table:

% marks	Percentage of students scoring the marks
10*	08
30	20
50	57
70	09
90	06

- As explained earlier, we expect a student picked up randomly, to score around 50% because more than half of the students have scored marks around 50%.

Mathematical Expectation

- Thus, we assign weight (8/100) to the score 10% (Therefore 8, out of 100 students, score on the average 10% marks); (20/100) to the score 30% and so on.
- Thus

$$\text{Expected \% of marks} = 10 \times \frac{8}{100} + 30 \times \frac{20}{100} + 50 \times \frac{57}{100} + 70 \times \frac{9}{100} + 90 \times \frac{6}{100} = 47$$

The Principle of Induction

Induction plays an important role to many facets of data structure and algorithms.

Mathematical Induction is a method of writing a mathematical proof generally to establish that a given statement is true for all natural numbers.

The Principle of Induction

- It consists of the following three major steps:
 - **Induction base-** In this stage we verify/establish the correctness of the initial value. It is the proof that the statement is true for $n = 1$ or some other starting value.
 - **Induction Hypothesis-** it is the assumption that the statement is true for any value of n where $n \geq 1$.
 - **Induction Step-** In this stage we make a proof that if the statement is true for n , it must be true for $n+1$.

Efficiency of an Algorithm

- The size of an instance of the problem under consideration and the role of size in determining complexity of the solution.
- For example finding the product of two 2×2 matrices will take much less time than the time taken by the same algorithm for multiplying say two 100×100 matrices.

Efficiency of an Algorithm

- There are different measures of size of an instance of a problem are used for different types of problem. Two examples are :
 - In sorting and searching problems, the number of elements, which are to be sorted or are considered for searching, is taken as the size of the instance of the problem of sorting/searching.
 - In the case of solving polynomial equations or while dealing with the algebra of polynomials, the degrees of polynomial instances, may be taken as the sizes of the corresponding instances.

Asymptotic Analysis & Notations

- Asymptotic analysis is a more formal method for analyzing algorithmic efficiency. It is a mathematical tool to analyze the time and space complexity of an algorithm as a function of input size.
- For example, when analyzing the time complexity of any sorting algorithm such as Bubble sort, Insertion sort and Selection sort in the worst .

Asymptotic Analysis & Notations

- Algorithms in the worst case takes $T(n) = n^2$ where n is a size of the list.

In contrast, Merge sort takes time

$$T(n) = n \cdot \log^2(n)$$

Asymptotic Analysis & Notations

- Some common orders of growth seen often in complexity analysis are

$O(1)$	constant
$O(\log n)$	logarithmic
$O(n)$	linear
$O(n \log n)$	$n \log n$
$O(n^2)$	quadratic
$O(n^3)$	cubic
$O(n^k)$	polynomial
$O(2^n)$	exponential

Asymptotic Analysis & Notations

- **Worst Case and Average Case Analysis**
 - The worst case algorithm is the element to be searched for is either not in the list or located at the end of the list.
 - In this case the algorithm runs for the longest possible time. It will search the entire list.
 - If an algorithm runs in time $T(n)$, means that $T(n)$ is an upper bound on the running time that holds for all inputs of size n . This is called worst-case analysis.

Asymptotic Analysis & Notations

- Average-case analysis which provides average amount of time to solve a problem. In which calculate the expected time spent on a randomly chosen input.
- This kind of analysis is generally more difficult compared to worst case analysis.

Asymptotic Analysis & Notations

- **Asymptotic Notations**
 - There are mainly three asymptotic notations
 - Big-O notation,
 - Big- Θ (Theta) notation
 - Big- Ω (Omega) notation

Asymptotic Analysis & Notations

- **Big-O notation: Upper Bounds**

- Big O is used to represent the upper bound or a worst case of an algorithm.
- It bounds the growth of the running time from above for large value of input sizes. It notifies that a particular procedure will never go beyond a specific time for every input n .
- One important advantage of big-o notation is that it makes algorithms much easier to analyze.

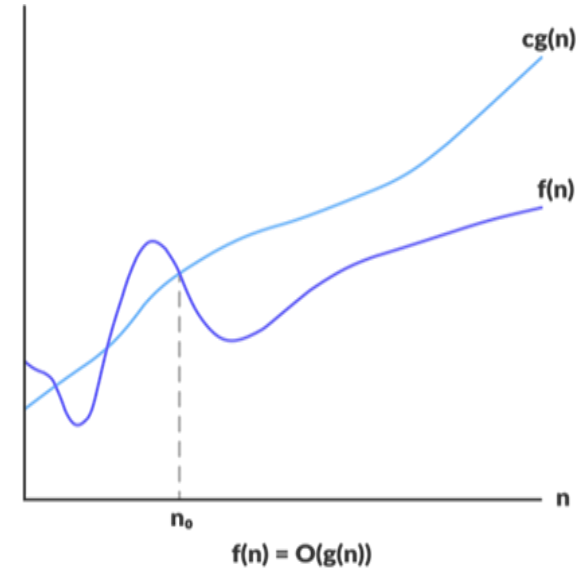
Asymptotic Analysis & Notations

- Big-Oh notation can be defined as follows-

$$F(n) = O(g(n))$$

$$F(n) \leq C \cdot g(n): \forall n \geq n_0$$

- When a running time of $f(n)$ is $O(g(n))$, it means the running time of a function is bounded from above for input size n by $c \cdot g(n)$



Asymptotic Analysis & Notations

- Verify the complexity of $3n^2 + 4n - 2$ is $O(n^2)$?
- In this case $f(n) = 3n^2 + 4n - 2$ and $g(n) = n^2$
- The above function is still a quadratic algorithm and can be written as:

$$\begin{aligned} 3n^2 + 4n - 2 &\leq 3n^2 + 4n^2 - 2n^2 \\ &\leq (3 + 4 - 2) n^2 \\ &= O(n^2) \end{aligned}$$

Asymptotic Analysis & Notations

- Now to find out what values of c and n_0 , so that

$$3n^2 + 4n - 2 \leq cn^2 \text{ for all } n \geq n^0.$$

- If n^0 is 1, then c must be greater than or equal to $3 + 4 - 2 \leq c$, 6.

So, above function can now be written as-

$$3n^2 + 4n - 2 \leq 6n^2 \text{ for all } n \geq 1$$

So, we can say :

$$3n^2 + 4n - 2 = O(n^2).$$

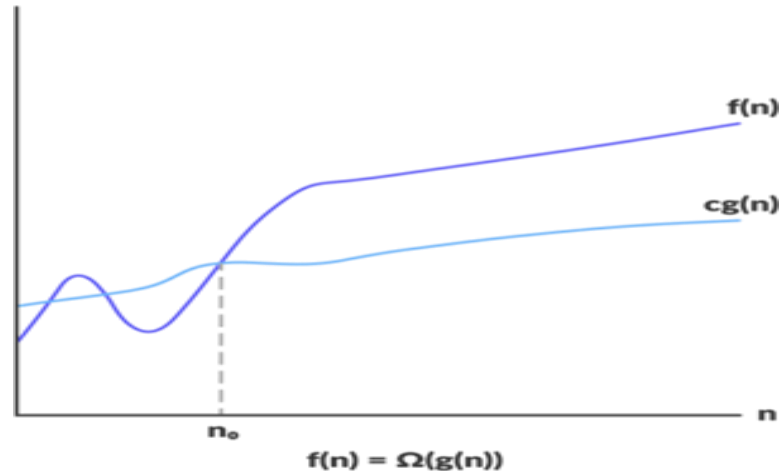
Asymptotic Analysis & Notations

- **Big-Omega (n) Notation**

- Big Omega describes the asymptotic lower bound of an algorithm whereas a big Oh(O) notation represents an upper bound of an algorithm.
- That an algorithm takes at least this amount of time without mentioning the upper bound.
- $f(n) = \Omega(g(n))$ if and only if there exists some constants C and n_0 such that $f(n) \geq C \cdot g(n) : n \geq n_0$. The following graph illustrates the growth of $f(n) = \Omega(g(n))$

Asymptotic Analysis & Notations

- Let's define it more formally:
 $f(n) = \Omega(g(n))$ if and only if there exists some constants C and n_0 such that $f(n) \geq C \cdot g(n) : n \geq n_0$. The following graph illustrates the growth of $f(n) = \Omega(g(n))$.



Asymptotic Analysis & Notations

- If $f(n)$ is $\Omega(g(n))$ which means that the growth of $f(n)$ is asymptotically no slower than $g(n)$ no matter what value of n is provided.

$$f(n) = \Omega(g(n))$$

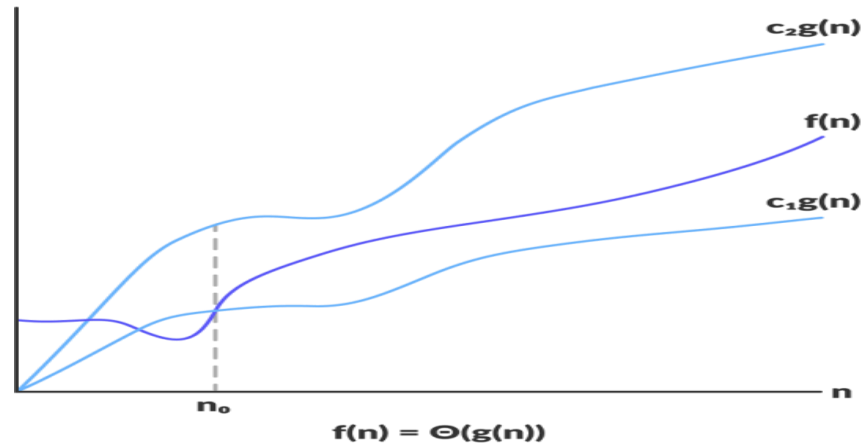
Asymptotic Analysis & Notations

Θ (Theta) notation: Tight Bounds

- The running time of an algorithm is $\theta(n)$, it means that once n gets
- large enough, the running time is minimum $c_1 \cdot n$, and maximum $c_2 \cdot n$, where c_1 and c_2 are constants. It provides both upper and lower bounds of an algorithm.

Asymptotic Analysis & Notations

- The following figure illustrates the function $f(n) = \Theta(g(n))$, where the value of $f(n)$ lies between $c_1(g(n))$ and $c_2(g(n))$ for sufficiently large values of n .



Asymptotic Analysis & Notations

- **Theorem:** For any two functions $f(x)$ and $g(x)$, $f(x) = O(g(x))$ if and only if $f(x) = O(g(x))$ and $f(x) = \Omega(g(x))$.
- If $f(n)$ is $\theta(g(n))$ this means that the growth of $f(n)$ is asymptotically at the same rate as $g(n)$ or we can say the growth $f(n)$ is not asymptotically slower or faster than $g(n)$ no matter what value of n is provided.

Asymptotic Analysis & Notations

- Some Useful Theorems for O , Ω , Θ

Theorem 1: If $f(n) = a_m n^m + a_{m-1} n^{m-1} + \dots + a_1 n + a_0$

Where $a_m \neq 0$ and $a_i \in R$, then $f(n) = O(n^m)$

Proof: $f(n) = a_m n^m + a_{m-1} n^{m-1} + \dots + a_1 n + a_0$

$$= \sum_{i=0}^m a_i n^i$$

$$f(n) \leq \sum_{i=0}^m |a_i| n^i$$

$$\leq n^m \sum_{i=0}^m |a_i| n^{i-m} \leq n^m \sum_{i=0}^m |a_i| \text{ for } n \geq 1$$

Let us assume $|a_m| + |a_{m-1}| + \dots + |a_1| + |a_0| = c$

$$\text{Then } f(n) \leq c n^m \Rightarrow f(n) = O(n^m).$$

Asymptotic Analysis & Notations

Theorem 2: If $f(n) = a_m n^m + a_{m-1} n^{m-1} + \dots + a_1 n + a_0$

Where $a_m \neq 0$ and $a_i \in R$, then $f(n) = \Omega(n^m)$.

Proof: $f(n) = a_m n^m + \dots + a_1 n + a_0$

Since $f(n) \geq c n^m$ for all $n \geq 1 \Rightarrow f(n) = \Omega(n^m)$

Asymptotic Analysis & Notations

Theorem 3: If $f(n) = a_m n^m + a_{m-1} n^{m-1} + \dots \dots \dots a_1 n + a_0$

Where $a_m \neq 0$ and $a_i \in R$, then $f(n) = O(n^m)$.

Proof: From Theorem 1 and Theorem 2,

$$f(n) = O(n^m) \dots \dots \dots (1)$$

$$f(n) = \Omega(n^m) \dots \dots \dots (2)$$

From (1) and (2) we can say that $f(n) = \Theta(n^m)$

Important Topics

Mathematical Functions & Notations

Summation & Product, Function, Logarithms

Mathematical Expectation

Asymptotic Functions & Notations

Principle of Mathematical Induction

Summary

- In this session we have seen that :-
- Solving any problem algorithm is designed. An algorithm is a definite, step-by-step procedure for performing some task.
- An algorithm takes some sort of input and produces some sort of output
- Algorithm must be efficient and easy to understand.
- There are three popular asymptotic notations namely, big O, big Ω , and big Θ .

*Thank
You*



Design and Analysis of Algorithms

Block -1 Unit-3

Complexity Analysis of Simple Algorithms

Topics to be Covered

- Introduction
- A Brief Review of Asymptotic Notations
- Analysis of Simple Constructs or Constant Time
- Analysis of Simple Algorithms
- Important Topics
- Summary

Introduction

- Computational complexity describes the amount of processing time required by an algorithm to give the desired result.
- The worst-case time complexity (big O notation) which is the maximum amount of time required to execute an algorithm for inputs of a given size.
- Whereas average case complexity, which is the average of the time taken on inputs of a given size is less common.

Asymptotic Notations

- These notations is to focus on only the time required to execute an algorithm & compare the relative rate of growth of functions.
- Assume $T(n)$ and $f(n)$ are two functions
 - $T(n) = O(f(n))$ if there are two positive constants C and n_0 such that $T(n) \leq Cf(n)$ where $n \geq n_0$
 - $T(n) = \Omega(f(n))$ if there are two positive constants C and n_0 such that $T(n) \geq C\Omega f(n)$ where $n \geq n_0$
 - $T(n) = \theta(f(n))$ if and only if $T(n) = O(f(n))$ and $T(n) = \Omega(f(n))$

Analysis of Simple Constructs or Constant Time

- **O(1):** Time complexity of a function (or set of statements) is considered as O(1) 'if (i) statements are simple statement like assignment, increment or decrement operation and declaration statement and (ii) there is no recursion, loops or call to any other function. Example -

int x;

x = x + 5

x = x - 5

Analysis of Simple Constructs or Constant Time

- **$O(n)$** : This is running time of a single looping statement which includes comparing time, increment or decrement by some constant value looping statement. Example -

```
for (i = 1; i <= n; i += c) {  
    // simple statement(s) }  
  
for (int i = n; i > 0; i -= c) {  
    // simple statement(s)  
}
```

Analysis of Simple Constructs or Constant Time

- **$O(n^c)$:** This is a running time of nested loops. Time complexity of nested loops is equal to the number of times the innermost statements is executed. For example, the following sample loops have $O(n^2)$ time complexity.

Example –

```
for (int i = 1; i<=n; i += c) {  
  
    for (int j = 1; j <=n; j += c) {  
  
        // some simple statements  
  
    }  
}
```

Analysis of Simple Constructs or Constant Time

- **$O(\log n)$** : If the loop index in any code fragment is divided or multiplied by a constant value, the time complexity of the code fragment is $O(\log n)$.

Example -

```
for (int i = 1; i <= n; i *= c ) {  
    // some simple statements }  
  
for (int i = n; i > 0; i /= c) {  
    // simple statements  
}
```

Analysis of Simple Constructs or Constant Time

- **Time complexities of consecutive loops :** If the code fragment is having more than one loop, time complexity of the fragment is sum of time complexities of the individual loops.

Example -

```
for (int i = 1; i<=m; i ++ c) {  
    // simple statements taking  $\theta(1)$   
} for (int f = 1; i<=n; if += X) {  
    // simple statements of  $\theta(1)$   
}
```

Analysis of Simple Constructs or Constant Time

- **Consecutive if else statements**
 - The running time of the if-else statement is just running time of the testing the condition plus the larger of the running of times statement1 or statement2

code fragment of
if – else is
if (condition)
statement 1 else
statement 2

Analysis of Simple Algorithms

- **A Summation Algorithm**

- The following is a simple program to calculate

```
int sum of n cube (int n)
{
    int i, temp result;
    Temp result =0;
    for (i=1 ; I <=n; i++)
    Temp result = temp result + i * i * i
    return tempresult;
}
```

Analysis of Simple Algorithms

- **Polynomial Evaluation**

- A polynomial is an expression that contains more than two terms. A term comprises of a coefficient and an exponent.

$$P(x) = 15x^4 + 7x^2 + 9x + 7 \quad P(x) = 14x^4 + 17x^3 - 12x^2 + 13x + 16$$

- A polynomial may be represented in form of array or structure.

Analysis of Simple Algorithms

- A structure representation of a polynomial contains two parts
 - coefficient
 - the corresponding exponent.
- The following is the structure definition of a polynomial:

Struct polynomial

{

int coefficient;

int exponent;

};

Analysis of Simple Algorithms

- **Analysis of Brute Force Method**

- A brute force approach to evaluate a polynomial is to evaluate all terms one by one.
- First calculate x^n , multiply the value with the related coefficient a_n , repeat the same steps for other terms ,then return the sum

$$p(x) = a_n \cdot x \cdot x \cdot \dots \cdot x \cdot x + a_{n-1} \cdot x \cdot x \cdot \dots \cdot x \cdot x + a_{n-2} \cdot x \cdot x \cdot \dots \cdot x \cdot x + \dots + a_2 \cdot x \cdot x + a_1 \cdot x + a_0$$

Analysis of Simple Algorithms

- **Analysis of Horner's Method**

In the first term it takes one multiplication, in the second term one multiplication and so on..

$$P(x) = (\dots(((a_n * x + a_{n-1}) * x + a_{n-2}) * x + \dots + a_2) * x + a_1) * x + a_0$$

Analysis of Simple Algorithms

- **Pseudo code for polynomial evaluation using Horner method, Horner(a,n,x)**
 - Assign value of poly $p[n]$ = coefficient of nth term in the polynomial
 - set $i=n-1$
 - compute $p = p * x + \text{poly}[i]$;
 - $i=i-1$
 - if i is greater than or equal to 0 Go to step4.
 - final polynomial value at x is p .

Analysis of Simple Algorithms

- **Step II**
 - Algorithm to evaluate polynomial at a given point x using Horner's rule:
 - **Input:** An array $A[0..n]$ of coefficient of a polynomial of degree n and a point x .
 - **Output:** The value of polynomial at given point x .

Analysis of Simple Algorithms

Evaluate_Horner (a,n,x)

{

p = A[n];

for (i = n-1; i $\geq$ 0;i--)

p = p * x + A[i];

return p;

}

Analysis of Simple Algorithms

- **Complexity Analysis**

- Polynomial of degree n using Horner's rule is evaluated as below:
- Initial assignment, $p = a[n]$
- After the first iteration $p = x a_n + a_{n-1}$
- After the second iteration, $p = x(x a_n + a_{n-1}) + a_{n-2} = x^2 a_n + x a_{n-1} + a_{n-2}$
- Every subsequent iteration uses the result of previous iteration i.e next iteration multiplies the previous value of p then adds the next coefficient.

Matrix (N x N) Multiplication

- Matrix is very important tool which is managing the data in matrix form it will be easy to manipulate and obtain more information. One of the basic operations on matrices is multiplication.

$$a_m, a_m + 1, a_m + 2, \dots, a_n$$

Matrix (N X N) Multiplication

- For Example multiply two square matrix of order $n \times n$ and find its time complexity.
- Multiply two matrices A and B of order $n \times n$ each and store the result in matrix C of order $n \times n$. A square matrix of order $n \times n$ is an arrangement of set of elements in n rows and n columns.

Matrix (N X N) Multiplication

- Matrix of order 3 x 3 which is represented as

$$A = \begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{pmatrix} \quad 3 \times 3 \text{ matrix}$$

Matrix (N X N) Multiplication

- **Step I:**

- Pseudo code: For Matrix multiplication problem where we will multiply two matrices A and B of order 3x3 each and store the result in matrix C of order 3x3.
 - Multiply first row first element of first matrix with first column first element of second matrix.
 - Similarly perform this multiplication for first row of first matrix and first column of second matrix. Now take the sum of these values.
 - The sum obtained will be first element of product matrix C

$$c_{11} = a_{11} \times b_{11} + a_{12} \times b_{21} + a_{13} \times b_{31}$$

$$C = A \times B$$

Matrix (N X N) Multiplication

- **Step I :**
 - Pseudo code: For Matrix multiplication problem where we will multiply two matrices A and B of order 3x3 each and store the result in matrix C of order 3x3.
 - Multiply first row first element of first matrix with first column first element of second matrix.

Matrix (N X N) Multiplication

- **Step II :**

- Algorithm for multiplying two square matrix of order $n \times n$ and find the product matrix of order $n \times n$
 - Input: Two $n \times n$ matrices A and B
 - Output: One $n \times n$ matrix $C = A \times B$

Matrix Multiply(A,B,C,n)

{

for i = 0 to n-1

for j = 0 to n-1{

C[i][j]=0 //loop

C[i][j] = C[i][j] + A[i][k] * B[k][j] } }

Matrix (N X N) Multiplication

- **Complexity Analysis**

- First step is, for loop that will be executed n number of times i.e. it will take $O(n)$ time.
- The second nested for loop will also run for n number of time and will take $O(n)$ time & constant time i.e. $O(1)$.

Matrix (N X N) Multiplication

- The third for loop i.e. innermost nested loop will also run for n number of times and will take $O(n)$ time . Assignment statement inside third for loop will cost $O(1)$ as it includes one multiplication
- Total time complexity of the algorithm will be $O(n^3)$ for matrix multiplication of order $n \times n$.

Matrix (N X N) Multiplication

- **Exponent Valuation**

- Exponent evaluation is the most important operation. It has applications in cryptography and encryption methods, The exponent tells us how many times to multiply the base by itself.
 - **Left to right binary exponentiation**
 - **Right to left binary exponentiation**

Linear Search

- Linear search is the simplest method for searching. In Linear search technique of searching; the element is searched sequentially in the list.
- This method can performed on a sorted or an unsorted list (usually arrays).
- In case of a sorted list searching starts from 0th element and continues until the element is found from the list or the element whose value is greater than (assuming the list is sorted in ascending order), the value being searched is reached.

Linear Search

- **Linear_Search(A[], X)**
 - Step 1: Initialize i to 1
 - Step 2: if i exceeds the end of an array then print “element not found” and Exit
 - Step 3: if $A[i] = X$ then Print “Element X Found at index i in the array” and Exit
 - Step 4: Increment i and go to Step 2

Linear Search

Case	Best Case	Worst Case	Average Case
item is present	$O(1)$	$O(n)$	$O(n/2) = O(n)$
item is not present	$O(n)$	$O(n)$	$O(n)$

Sorting

- Sorting is the process of arranging a collection of data into either ascending or descending order.
 - **Internal Sort:** - Internal sorts are the sorting algorithms in which the complete data set to be sorted is available in the computer's main memory.
 - **External Sort:** - External sorting techniques are used when the collection of complete data cannot reside in the main memory but must reside in secondary storage for example on a disk.

Sorting

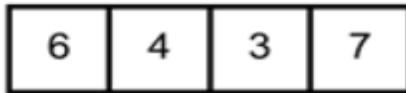
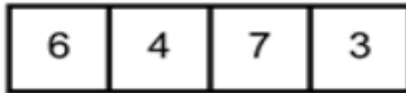
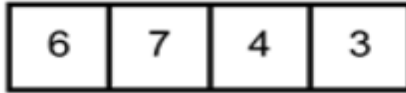
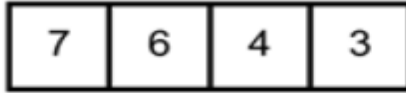
- **Bubble Sort**

- It is the simplest sorting algorithm in which each pair of adjacent elements is compared and exchanged if they are not in order.
- This algorithm is not recommended for use for a bigger size array .
- Largest element in the given unsorted array, bubbles up towards the last place in every cycle/pass .

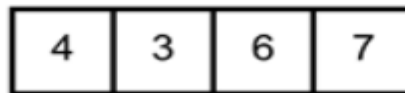
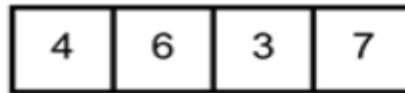
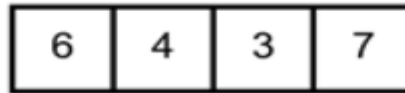
Sorting

Bubble Sort

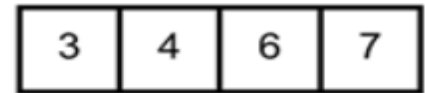
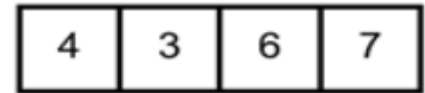
First pass



Second pass



Third pass



Sorting

- **bubble_sort (A,n)**

```
{  
  int i,j  
  for ( i= 1 to n-1 for (j = 0 to n-2  
    {  
      if (A[j]>A[j+1])  
      {  
        // swapping of two adjacent elements of an array A exchange (A[j],  
        A[j+1])  
      }  
    }  
  }
```

Important Topics

- Explain asymptotic notations
- Analysis of simple constructs or constant time
- Summation algorithm
- Polynomial evaluation algorithm
- Exponent evaluation
- Sorting algorithm

Summary

- In this session after making a brief review of asymptotic notations, complexity analysis of simple algorithms in illustrated simple summation, matrix multiplication, polynomial evaluation, searching and sorting.
- Horner's rule is discussed to evaluate the polynomial and its complexity is $O(n)$.
- Basic matrix multiplication is explained for finding product of two matrices of order $n \times n$ with time complexity in the order of $O(n^3)$. For exponent evaluation both approaches i.e left to right binary exponentiation and right to left binary exponentiation is illustrated. Time complexity of these algorithms to compute x^n is $O(\log n)$.
- Different versions of bubble sort algorithm are presented and its performance analysis is done at the end.

*Thank
You*



Design and Analysis of Algorithms

Block-1 Unit-4

Solving Recurrences

Topics to be Covered

- Introduction
- Recurrence Relation
- Methods for Solving Recurrence Relation
- Important Topics
- Summary

Introduction

- Complexity analysis of iteration algorithms is much easier as compared to recursive algorithms. But, once the recurrence relation/equation is defined for a recursive algorithm, which is not difficult task, then it becomes easier task to obtain the asymptotic bounds (Θ , O) for the recursive solution.
- In this unit we focus on recursive algorithms exclusively. Three techniques for solving recurrence equation are discussed: (i) Substitution method (ii) Recursion Tree Method and Master Method.

Recurrence Relation

- Recurrence relation to describe the running time of a recursive algorithm. A recursive algorithm can be defined as an algorithm which makes a recursive call to itself with smaller data size.
- Recursively, especially those problems which are solved through divide and conquer technique. The main problem is divided into smaller sub-problems which are solved recursively.
 - Merge Sort, Binary search, Strassen's multiplication algorithm are formulated as recursive algorithms .

Methods for Solving Recurrence Relations

- **Substitution Method**

- Substitution is opposite of induction .We start at n and move backward. A substitution method is one, in which we guess a bound and then use mathematical induction to prove whether our guess is correct or not. It comprises two steps:
 - Step1: Guess the asymptotic bound of the Solution.
 - Step2: Prove the correctness of the guess using Mathematical Induction

Methods for Solving Recurrence Relations

- Example :-** A Fibonacci sequence $f_0, f_1, f_2, \dots$ can be defined by the recurrence relation as:

$$f_n = \begin{cases} 0 & \text{if } n = 0 \\ 1 & \text{if } n = 1 \\ f_{n-1} + f_{n-2} & \text{if } n \geq 2 \end{cases}$$

- (Basic Step) The given recurrence relation has two base cases: $f_0 = 0$ and $f_1 = 1$. These two conditions (or values) where recursion does not call itself is called an initial condition (or Base conditions).
- (Recursive step): This step is used to find new terms $f_2, f_3, \dots$, from the existing (preceding) terms, by using the formula $f_n = f_{n-1} + f_{n-2}$ for $n \geq 2$.
- This formula says that “by adding two previous sequence (or term) we can get the next term”.
- For example $f_2 = f_1 + f_0 = 1 + 0 = 1$;

Methods for Solving Recurrence Relations

Example 1. Solve the following recurrence by using substitution method.

$$T(n) = 2T\left(\frac{n}{2}\right) + n$$

Solution: step1: The given recurrence is quite similar to that of Merge Sort algorithm, Therefore, our guess to the solution is $T(n) = O(n \log n)$

Or $T(n) \leq c \cdot n \log n$

Step2: Now we use mathematical Induction.

Here our guess does not hold for $n=1$ because $T(1) \leq c \cdot 1 \log 1$
i. e. $T(n) \leq 0$ which is contradiction with $T(1) = 1$

Now for $n=2$
 $T(2) \leq c \cdot 2 \log 2$

Methods for Solving Recurrence Relations

$$2T\left(\frac{n}{2}\right) + 2 \leq c \cdot n$$

$$2(1) + 2 \leq c \cdot 2$$

$$0 + 2 \leq c \cdot 2$$

$2 \leq c \cdot 2$ which is true. So $T(n) \leq c \cdot n \log n$ is True for $n = 2$

So

$T(n) \leq c \cdot n \log n$ is True for $n = 2$ |

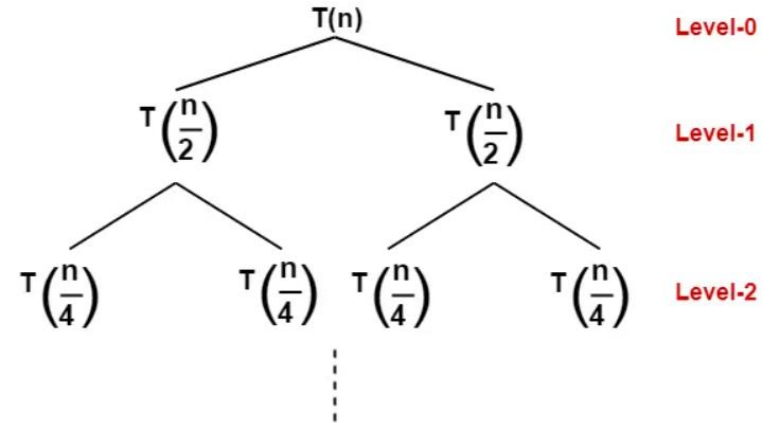
Methods for Solving Recurrence Relations

- **Recursion Tree method**

- Recursion tree method is especially used to solve a recurrence of the form:

$$T(n) = aT\left(\frac{n}{b}\right) + (n) \dots \dots \dots (1)$$

where $a > 1, b \geq 1$



https://www.gatevidyalay.com/recursion-tree-solving-recurrence-relations/#google_vignette

Methods for Solving Recurrence Relations

$$2T\left(\frac{n}{2}\right) + 2 \leq c \cdot n$$

$$2(1) + 2 \leq c \cdot 2$$

$$0 + 2 \leq c \cdot 2$$

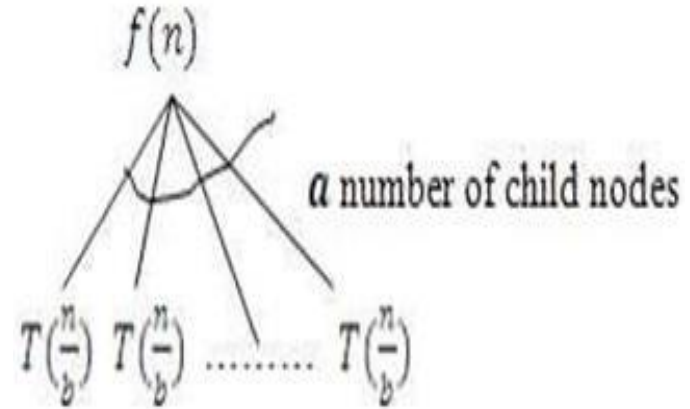
$2 \leq c \cdot 2$ which is true. So $(2) \leq c \cdot n \log n$ is True for $n = 2$

So

$(2) \leq c \cdot n \log n$ is True for $n = 2$ |

Methods for Solving Recurrence Relations

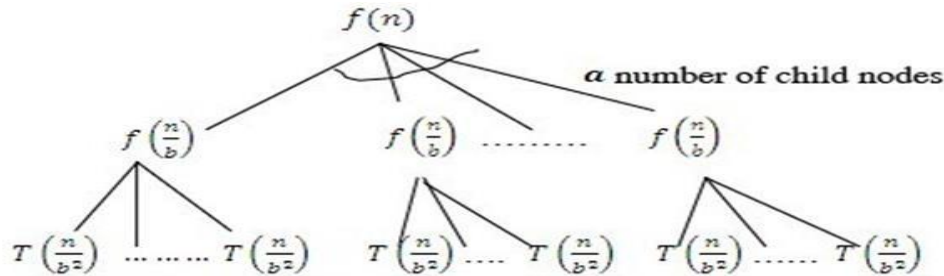
- Method (steps) for solving a recurrence $(n) = T(n/b) + (n)$ using recursion tree.
- We make a recursion tree for a given recurrence as follows:**
 - To make a recursion tree of a given recurrence (1), First put the value of $f(n)$ at root node of a tree and make a number of child nodes of this root value $f(n)$
Now tree will be looks like as:



Methods for Solving Recurrence Relations

- Now we have to find the value of $T(n)$ by putting (n/b) in place of n in **bequation** .That is

$$T\left(\frac{n}{b}\right) = aT\left(\frac{n}{b^2}\right) + f(n/b) = aT + f\left(\frac{n}{b^2}\right) + f(n/b) \dots (2)$$



7138 ,
u.in

Methods for Solving Recurrence Relations

- **Master Method**
- A function $f(n)$ is asymptotically positive if and only if there exists a real number n such that $f(x) > 0$ for all $x > n$.
- The master method provides us a straight forward method for solving recurrences of the form $T(n) = aT(n/b) + f(n)$, where $a > 1$ and $b > 1$ are constants and $f(n)$ is Asymptotically positive function. This recurrence gives us the running time of an algorithm that divides a problem of size n into a sub problems of size n/b . The a sub problems are solved recursively, each in time $T(n/b)$.

Methods for Solving Recurrence Relations

- Theorem1: Master Theorem**

- The Master Method requires memorization of the following 3 cases; then the solution of many recurrences can be determined quite easily, often without using pencil & paper.

Let $T(n)$ be defined on the non-negative integers by:
 $T(n) = aT\left(\frac{n}{b}\right) + f(n)$, where $a \geq 1, b > 1$

Then $T(n)$ can be bounded asymptotically as follows:

Case1: If $f(n) = O(n^{\log_b a - \epsilon})$ for some $\epsilon > 0$, then $T(n) = \Theta(n^{\log_b a})$

Case2: If $f(n) = \Theta(n^{\log_b a})$, then $T(n) = \Theta(n^{\log_b a} \cdot \log n)$.

Case3: If $f(n) = \Omega(n^{\log_b a + \epsilon})$ for some $\epsilon > 0$, and if $n^c f(n)$ for some constant $c < 1$ and all sufficiently large n . $T(n) = \Theta(f(n))$

Important Topics

- Substitution Method with theorem
- Recursion-tree Method with theorem
- Master Theorem with theorem

Summary

- In this session we have seen that when an algorithm contains a recursive call to itself, its running time can often be described by a recurrence equation which describes a function in terms of its value on smaller inputs.
- There are three basic methods of solving the recurrence relation:
 - The Substitution Method
 - The Recursion-tree Method
 - The Master Theorem

Thank
You



Design and Analysis of Algorithms

Block-2 Unit-1

Greedy Techniques

Topics to be Covered

- Introduction
- Example to Understand Greedy Techniques
- Formalization of Greedy Techniques
- An overview of Local and Global Optima
- Fractional Knapsack Problem
- A Task Scheduling Algorithm
- Huffman Codes
- Important Topics
- Summary

Introduction

- **Greedy Algorithm** a set of resources are recursively divided based on the maximum, immediate availability of that resource at any given stage of execution.
- To solve a problem based on the greedy approach, there are two stages Scanning the list of items Optimization.
- These stages are covered parallelly in this Greedy algorithm tutorial, on course of division of the array.

Examples to Understand Greedy Techniques

- **Example 1:**

- Suppose there is a list of tasks along with the time taken by each task. However, you are given with limited time only. The problem is which set of tasks will you be doing so that you can complete maximum number of tasks in the given amount of time.

- **Solution:**

- The intuitive approach would be greedy approach and the task selection criteria would be to select the task with the slowest amount of time. So, the first task will be that task which takes minimum time. The next task would be the one that takes minimum time among the remaining set of tasks and so on.

Examples to Understand Greedy Techniques

- **Example 2:**

- Consider a bus which can travel up to 40 kilometers (Km) with full tank. We need to travel from location 'A' to location 'B' which has distance of 95 Km as depicted in Figure



- In between 'A' and 'B', there are four gas stations, G1, G2, G3, and G4, which are at distance of 20 KM, 37.5 KM, 55 KM, and 75 KM, from location 'A' respectively. The problem is to determine the minimum number of refills needed to reach the location 'B' from location 'A'.

Examples to Understand Greedy Techniques

- Solution: Suppose, the tank refill decision criteria is considered to refill at the gas station which is nearest when the tank is about to get empty.
- According to the criteria, if we start from location 'A', the tank will be refilled at the second gas station (G2) as it is 37.5 KM from 'A'.
- After this, we need to refill at the fourth gas station (G4) which is at a distance of 37.5 KM from G2.
- From G4, we can easily reach the location 'B' as it is 20 KM. Therefore, the number of refills required is two as represented in the Figure.

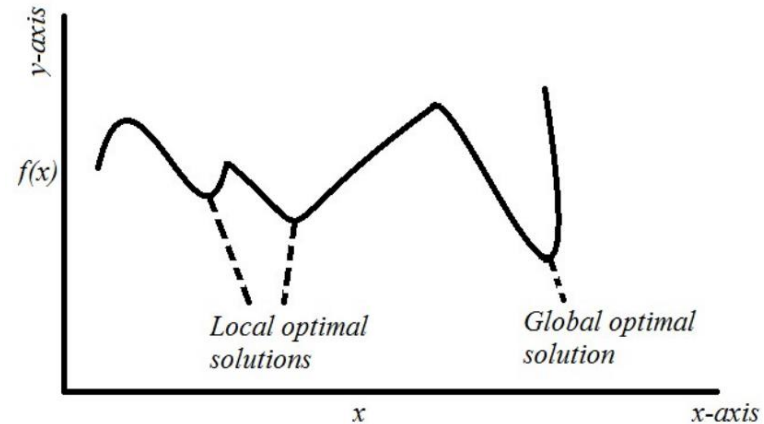


Formalization of Greedy Technique

- For a given problem with 'n' input values, the greedy approach will decide criteria to select values, termed as selection criteria, and will run for 'n' times. In each run, following steps will be performed:
 - It will perform selection based upon the selection criteria. This returns a value from the considered input values and also removes the selected value from the input values.
 - It will check the feasibility of the selected value.
 - If the solution is feasible then the selected value is added to the solution set else step 1 is repeated.

An Overview of Local And Global Optima

- Local optimization involves finding the optimal solution for a specific region of the search space, or the global optima for problems with no local optima.
- Global optimization involves finding the optimal solution on problems that contain local optima.



Fractional Knapsack Problem

- This is an optimization problem which we want to solve it through greedy technique. In this problem, a Knapsack (or bag) of some capacity is considered which is to be filled with objects.
- Example :- Given the weights and profits of N items, in the form of {profit, weight} put these items in a knapsack of capacity W to get the maximum total profit in the knapsack. In Fractional Knapsack, we can break items for maximizing the total value of the knapsack.

Input: $arr[] = \{\{60, 10\}, \{100, 20\}, \{120, 30\}\}, W = 50$

Output: 240

Explanation: By taking items of weight 10 and 20 kg and $\frac{2}{3}$ fraction of 30 kg.
Hence total price will be $60+100+(\frac{2}{3})(120) = 240$

Input: $arr[] = \{\{500, 30\}\}, W = 10$

Output: 166.667

Fractional Knapsack Problem

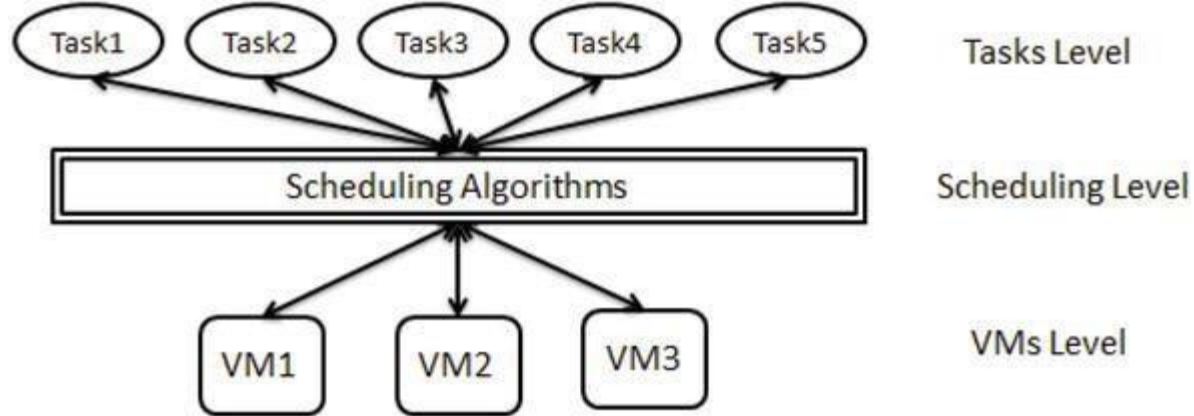
- The fractional knapsack problem is defined as:
- Given a list of n objects say $\{O_1, O_2, \dots, O_n\}$ and a Knapsack (or a bag).
- Capacity of Knapsack is W
- Each object O_i has a w_i weight and a profit of p_i
- If a fraction x_i (where $x_i \in \{0, \dots, 1\}$) of an object O_i is placed into a knapsack then a profit of $p_i x_i$ is earned.

Fractional Knapsack Problem

```
1: for  $i \leftarrow 1$  to  $n$  do
2:  $X[i] \leftarrow 0$ 
3: profit  $\leftarrow 0$  /*Total profit of items packed in Knapsack
4: weight  $\leftarrow 0$  /*Total weight of items packed in KnapSack
5:  $i \leftarrow 1$ 
6: while (weight  $\leq W$ ) //  $W$  is the Knapsack Capacity
{
7: if (weight +  $w[i] \leq W$ )
8:  $X[i] = 1$ 
9: weight = weight +  $w[i]$ 
10: else
11:  $X[i] = (W - \text{weight}) / w[i]$ 
12: weight =  $M$ 
13: profit = profit +  $p[i] * X[i]$ 
14:
} end of while
} //end of Algorithm
```


Task Scheduling Algorithm

- A task scheduling problem is formulated as an optimization problem in which we need to determine the set of tasks from the given tasks that can be accomplished within their deadlines along with their order of scheduling such that the profit is maximum.



Task Scheduling Algorithm

- Example : Let us consider an example to understand. Suppose, there are 5 tasks and each task has associated profit in rupees. The amount of time required to complete each task is one hour. However, the deadline of each task is given in specified in Table

Task	T ₁	T ₂	T ₃	T ₄	T ₅
Profit	18	22	5	7	6
Deadline	2	2	3	1	3

Task Scheduling Algorithm

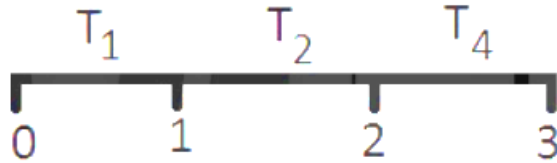
- Solution: To solve this problem, consider the highest deadline and prepare that many number of slots of unit time to schedule the tasks. This will help in scheduling the tasks easily as there will be no task to be completed after the highest deadline. According to the given problem, the highest deadline is 3. So, there will be three slots as illustrated in Figure, each slot of unit time.



- Now, greedy approach has to select the set of three tasks and schedule them in such a way that the total profit is maximum on their completion.

Task Scheduling Algorithm

- The schedule of given tasks is presented in Figure

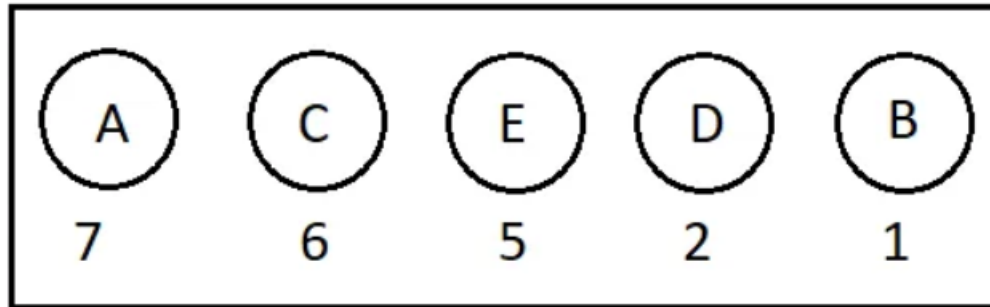
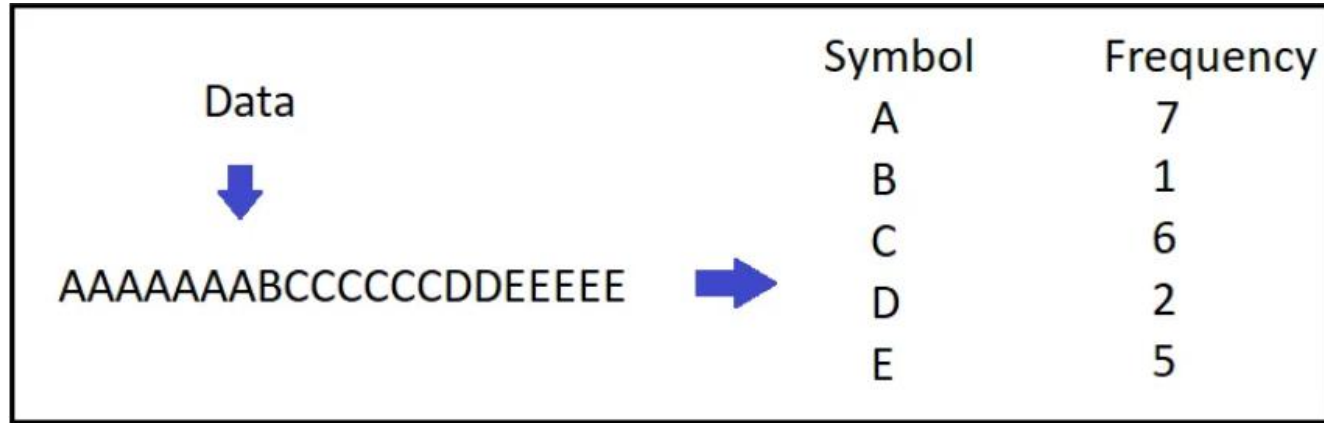


- Therefore, the sequence of selected tasks is as follows:
- {T₁, T₂, T₄} The total profit is: $18+22+7= 47$

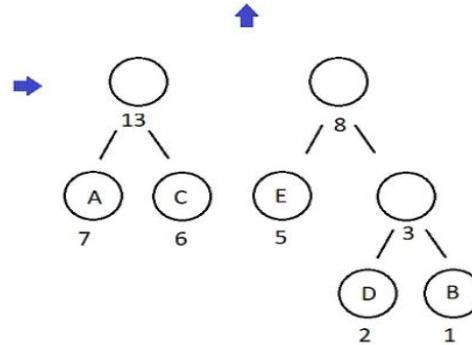
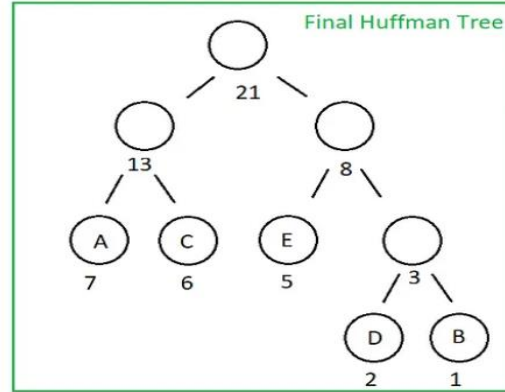
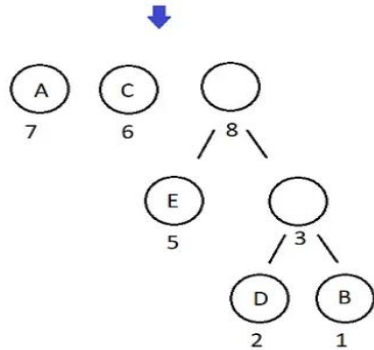
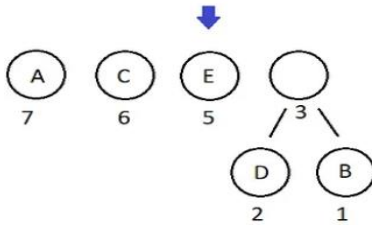
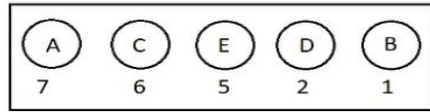
Huffman Codes

- Huffman coding is a greedy algorithm that is used to compress the data. Data can be sequence of characters and each character can occur multiple times in a data called as frequency of that character.
- Data transmitted through transmission media that can be digital communication or analog communication & represent data in the form of bits.
- Huffman compression algorithm is applied to represent the data in compressed form called as Huffman codes.

Huffman Codes



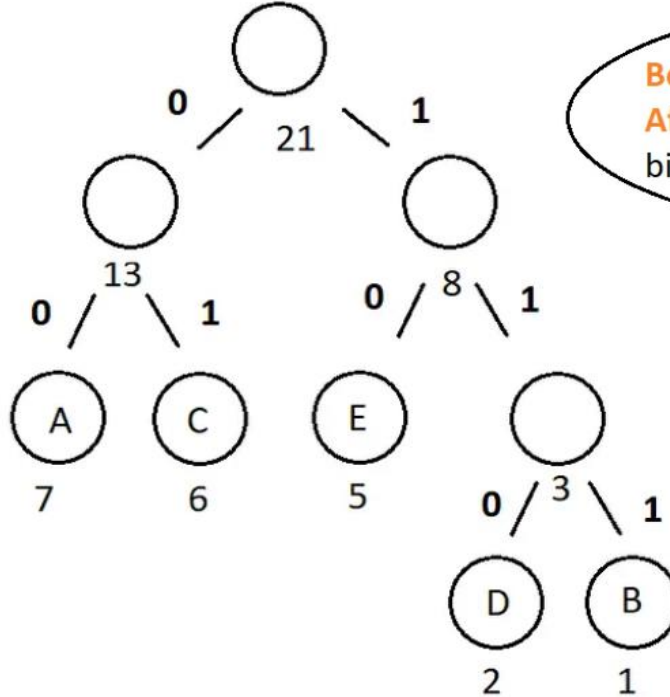
Huffman Codes



Huffman Codes

Symbol Bit Code

A	00
B	111
C	01
D	110
E	10

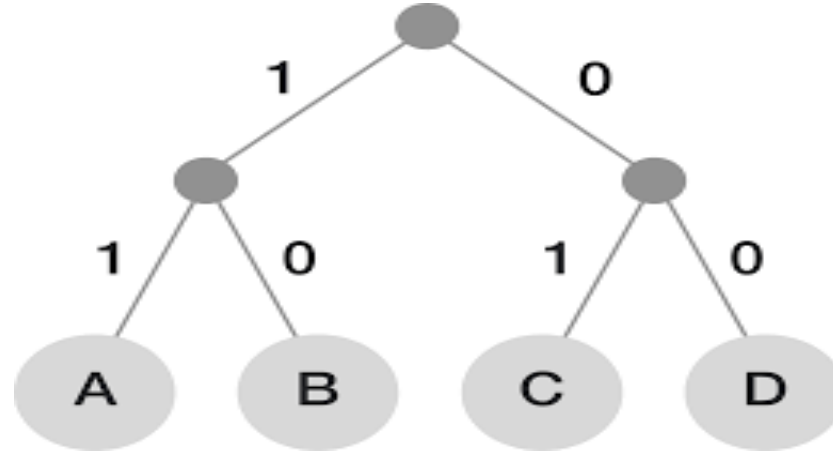


Before compression = $21 \times 8 \text{ bits} = 198 \text{ bits}$

After compression = $7 \times 2 \text{ bits} + 1 \times 3 \text{ bits} + 6 \times 2 \text{ bits} + 2 \times 3 \text{ bits} + 5 \times 2 \text{ bits} = 45 \text{ bits}$

Huffman Codes

- Huffman coding works in two steps:
 - Build a Huffman tree.
 - Find Huffman code for each character



Huffman Codes

- **Steps for Building Huffman Tree:**
 - Input is a set of M characters and each character $m \in M$ has a frequency $mfrequency$
 - Store all the characters in min-priority queue using frequencies as their values (Priority queue is a data structure that allows the operation of search min (or max), insert, delete min (or max, respectively).
 - If heap is used to implement priority queue , it will take $O(\log n)$ time.

Huffman Codes

- Extract two minimum nodes x , y from min-priority queue PQ .
- Replacing x , y in the queue with a new-node z representing their merger.
The frequency of z is computed as sum of the frequencies of x and y . The node z has x as its left child and y as its right child.
- Repeat steps 3 and 4 until one node left in the queue, which is the root of the Huffman tree.
- Return the root of the tree.

Huffman Codes

- **Steps to find Huffman code for each character**
 - Traverse the tree starting from the root node.
 - An edge connecting to the left child node is labeled as 0 and 1 if it is connecting to the right child node.
 - Traverse the complete tree through the left and right child based on the assigned value.
 - Prefix code/ Huffman code for a letter is the sequence of labels on the edge connecting the root to the leaf for that letter.

Huffman Codes

- **Time Complexity of Huffman Algorithm**
 - Priority queue is implemented using binary-min heap. Building min heap procedure takes $O(n)$ time in step 2. Steps 3 and 4 run exactly $n-1$ times.
 - Since in each step a new node z is added in the heap. When new node added it take $O(\log n)$ time to heapify.
 - Thus, total running time of Huffman algorithm on a set of n characters is $O(n \log n)$.

Important Topics

- Formalization of Greedy Techniques
- Local and Global Optima
- Fractional Knapsack Problem
- Task Scheduling Algorithm
- Huffman Codes

Summary

- In this session we have seen that an optimization problem is a problem in which the best solution among all the possible solutions of a problem is needed which maximizes/minimizes the objective function under some constraints or conditions.
- Local optimal solutions correspond to the set of all the feasible solutions that are best locally for an optimization problem. Global optimal solution corresponds to the best solution of the optimization problem.
- In Fractional Knapsack problem, a Knapsack (or bag) of some capacity is to be filled with objects which can be included in fractions. Each object is given with a weight and associated profit.

*Thank
You*



Design and Analysis of Algorithms

Block-2 Unit-2

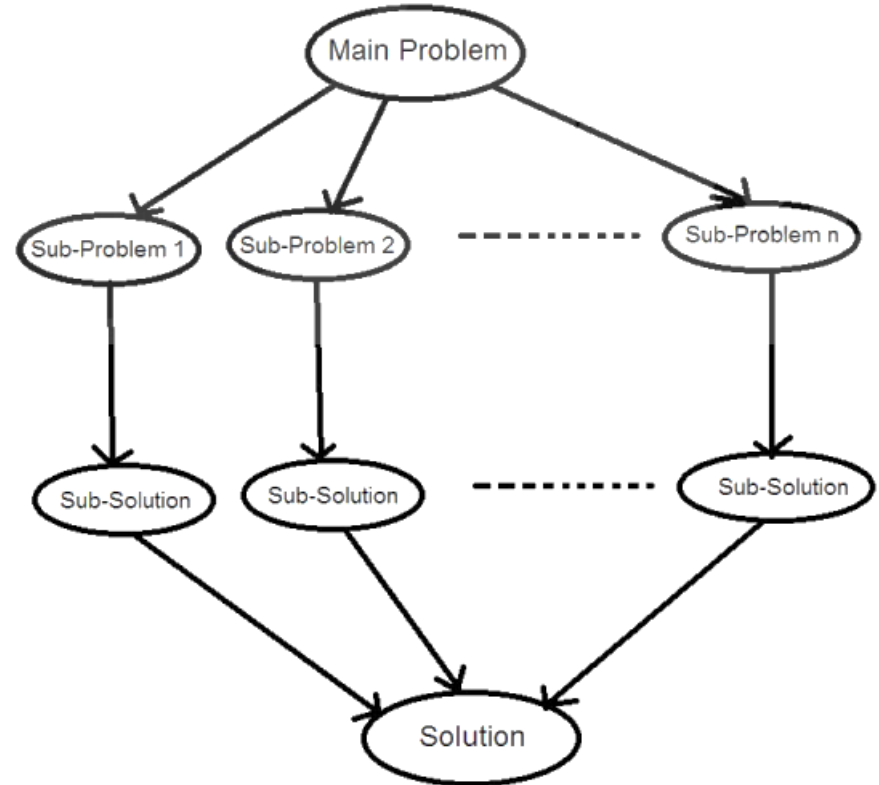
Divide & Conquer Technique

Topics to be Covered

- Introduction
- Recurrence Relation Formulation in Divide and Conquer Technique
- Binary Search Algorithm
- Sorting Algorithms
- Integer Multiplication
- Matrix Multiplication Algorithm
- Important Topics
- Summary

Introduction

- In Divide and Conquer approach, the original problem is divided into two or more sub-problems recursively.
- A **divide and conquer algorithm** is a strategy of solving a large problem by breaking the problem into smaller sub-problems solving the sub-problems, and combining them to get the desired output.



Recurrence Relations Formulations in Divide and Conquer Approach

- Divide and Conquer approach the running time is equated as a recurrence relation which is based upon three steps:
 - **Divide:** Dividing the given problem into sub-problems in such a way that each sub-problem is equivalent to the original problem but its size is smaller than the original one. Further sub-division of each sub-problem is done till either it is directly solvable or it is impossible to perform sub-division which indicates that there is a direct solution of the sub-problem.
 - **Conquer:** Each sub-problem solves itself by calling itself recursively.
 - **Combine:** Each solution of the sub-problem is combined to obtain the original solution.

Recurrence Relations Formulations in Divide and Conquer Approach

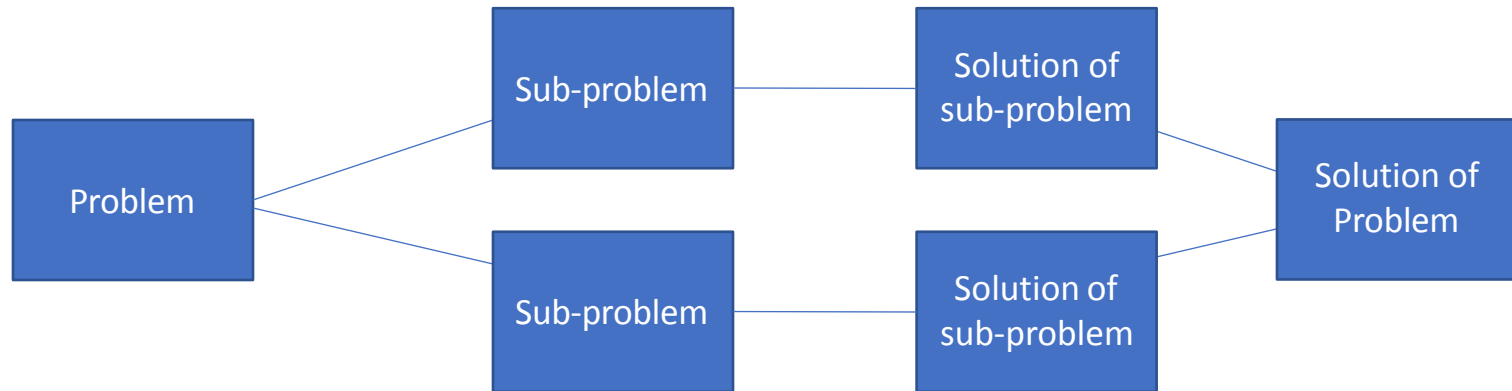
- where,

$$T(n) = \begin{cases} \theta(1) & \text{if } n \leq C \\ aT(n/b) + f(n) & \end{cases}$$

- $T(n)$ = time required to solve a problem of size 'n'.
- A corresponds to the number of partitions made to a problem.
- $T(n/b)$ corresponds to the running time of solving each sub-problem of size (n/b) .

Recurrence Relations Formulations in Divide and Conquer Approach

- $f(n) = D(n) + C(n)$ – time required to divide the problem and combine the solutions respectively. If the problem size is small enough say, $n \leq C$ for some constant C , we have a best case which can be directly solved in a constant time: $\theta(1)$, otherwise, divide a problem of a size n into sub problems, each of $1/b$ size.



Binary Search

- Binary search is a procedure of finding the location of an element in a sorted array.. The divide and conquer version of the algorithm proceeds as follows:
 - If the key is found at the middle position of the array, the algorithm terminates, otherwise
 - Divide the array into two sub-arrays recursively. If the key is smaller than the middle value, select the left part of the sub-arrays, otherwise (if it is larger) select the right part of :the sub-array. The process continues until the search interval is not empty or it cannot be broken further.

Binary Search

- Conquer (solve) the sub-array by determining whether the key is located in that sub-array.
- Obtain the result.
- Binary search looks for a particular item by comparing the middle most item of the collection. If a match occurs, then the index of item is returned. If the middle item is greater than the item, then the item is searched in the sub-array to the left of the middle item.

$$\text{mid} = \text{low} + (\text{high} - \text{low}) / 2$$

Binary Search

- **Analysis of Binary Search:**

Method1: the size of the array is a power of 2, say 2^k . Each time in the while loop, when we examine the middle element, we cut the size of the sub-array into half. So Before the 1st iteration size of the array is 2^k .

- After the 1st iteration size of the sub-array of our interest is: 2^{k-1}
- After the 2nd iteration size of the sub-array of our interest is: 2^{k-2}
- After the k^{th} iteration size of the sub-array of our interest is: $2^{k-k}=1$
- So we stop after the next iteration. Thus we have at most $(k+1) = (\log n + 1)$ iterations.

Binary Search

- **Recurrence Relation of Binary Search**
 - The complexity of divide and conquer approach is defined by recurrence relation as follows:
 - $T(n) = a T(n/b) + f(n)$
 - As binary search follows divide and conquer approach and performs dividing the array but searching on only one sub-array in iteration, the computational complexity of binary search technique can be defined as recurrence relation in the following form:
 - $T(n) = T(n/2) + k$

Binary Search

- Where, a , b , and $f(n)$ are replaced with values 1, 2, k (constant less than n), respectively. K is constant value for divide and conquer operation on solving this recurrence relation by substitution method, the computational complexity for binary search is $O(\log n)$.



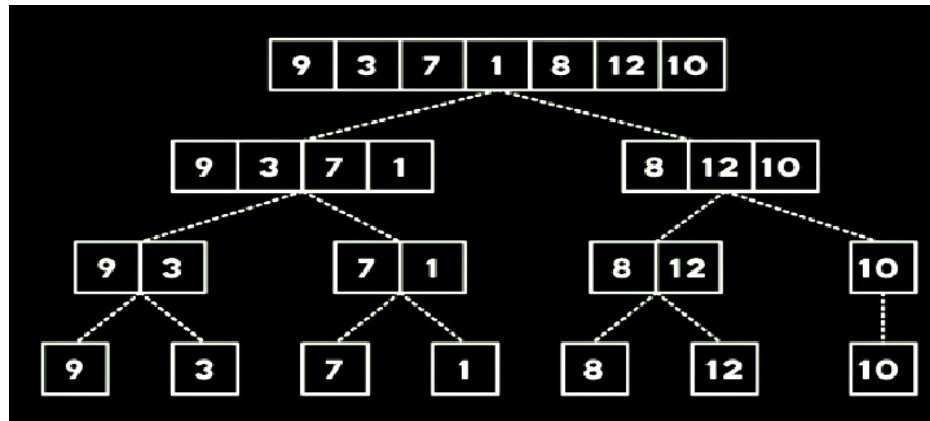
Sorting Algorithms

- It is process of rearranging the elements of a list in either ascending or descending order. For example, consider a list of elements $\langle a_1, a_2, a_3, \dots, a_n \rangle$ as input which needs to be sorted in ascending order, then the output of the sorting algorithm will be rearranging the list such that $\langle a_1 \leq a_2 \leq a_3 \leq \dots \leq a_n \rangle$. there are number of sorting algorithms like bubble sort, merge sort, radix sort, quick sort and many more.
- Merge-Sort and Quick-Sort sorting algorithms as they are based on divide and conquer approach.

Sorting Algorithms

- **Merge-Sort**

- Merge-Sort algorithm is a divide-and-conquer based sorting algorithm. It follows a divide and conquer approach to perform sorting. The algorithm in divide step repeatedly partitions an array into several sub arrays until each sub array consists of a single element.



Sorting Algorithms

- The following example illustrates the above steps: Suppose the array has following numbers:

37 20 23 30 18 15 27 17

In divide step , the array is divided into two sub arrays

37 20 23 30 and 18 15 27 17

In conquer step , we sort each sub array 20 23 30 37 and 15 17 18 27 In combine(merge) ,all the sub arrays are merged in sorted order 15 17 18 20 23 27 30 37

Sorting Algorithms

- **Algorithm 2:** Merge-Sort (X, m, n) if ($m < n$) {
 $i = (m+n)/2$;
 Merge-Sort (X, m, i); Merge-Sort ($X, i+1, n$);
 Merge (X, m, i, n);
}
 else {
 Already sorted
 }

Sorting Algorithms

- **Quick-Sort**

Quick-Sort is another sorting algorithm that works on the principle of divide- and-conquer approach. It works by arranging the elements in an array by identifying their correct place (or index).



Sorting Algorithms

- **Divide:** In this, an element of the given array is considered as pivot element whose correct index 'q' in the array is determined by rearranging the array elements. Then, the given array $A[m \dots n]$ is partitioned into two sub-arrays, $A[p..q]$ and $A[q+1..r]$, in such a way that all the elements in $A[p..q]$ are smaller than $A[q]$ and all the elements in $A[q+1..r]$ are greater than $A[q]$. Generally, the procedure which is used to perform this step is termed as Partition. The output of this procedure is the index 'q' which divides the given array 'A'.

Sorting Algorithms

- **Conquer:** The partitioned sub-arrays, $A[p..q]$ and $A[q+1..r]$, recursively call the step (1) to perform sorting of the elements.
- **Combine:** As all the elements are sorted in their respective places, there is no need to perform combine step and the array is sorted.

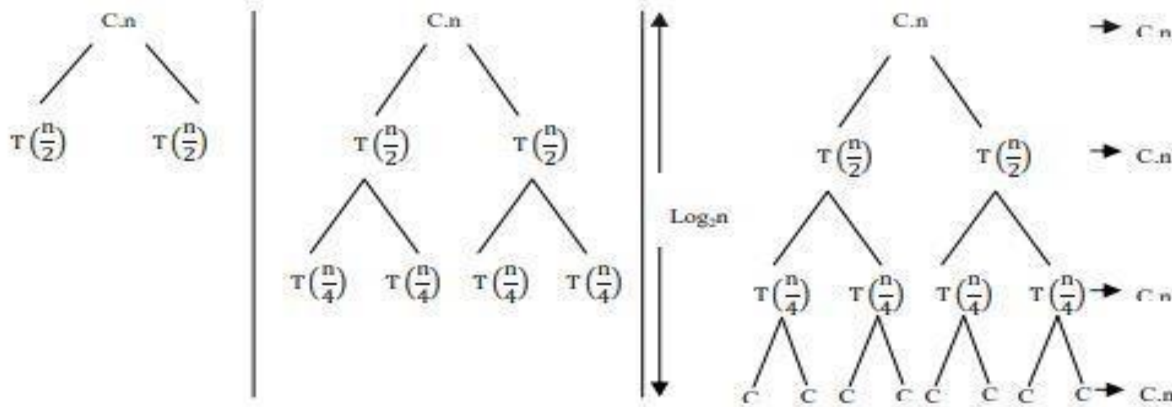


Sorting Algorithms

- **Analysis of Quick-Sort:**
 - The computation complexity of Quick-Sort is based on the arrangement of array elements. Arrangement of elements effects the partitioning of an array. If partitioning is unbalanced, partitioning procedure will perform more number of times in comparison to balanced partition. Therefore, the Quick-Sort has different complexity in different scenarios:
 - **Best Case:** If the input data is not sorted, then the partitioning of subarray is balanced. (i.e. $O(n \log n)$)

Sorting Algorithms

- **Worst Case:** If the given input array is already sorted or almost sorted, then the partitioning of the subarray is unbalancing in this case the algorithm runs asymptotically as slow as Insertion sort(i.e. $O(n^2)$).
- **Average Case:** Except best case or worst case. The shows the recursion depth of Quick-sort for Best, Worst and Average cases.



Integer Multiplication

- The brute force algorithm for multiplying two large integer numbers which everyone of us uses by hand, takes quadratic time i.e., $O(n^2)$. Because each digit of one number is multiplied by each digit in another number.
- Assume X and Y are two n digits number. Divide X and Y into two halves of approximately $n/2$ digits each.
- The following examples illustrate the division process:
 - $657,138 = 657 * 103 + 138$
 - $6578,381 = 6578 * 103 + 381$

Integer Multiplication

- Let us generalize the number representation. If Z is an n -digit number, it would be divided into two halves, the first half with Ceiling with $\lceil n/2 \rceil$ and the second half with Floor $\lfloor n/2 \rfloor$ as shown below:
 - $Z(\text{ n digit number}) = X_L * 10^m + X_R$
 - Suppose are given two n - digit numbers:
 - $Z1 = X_L * 10^m + X_R$
 - $Z2 = Y_L * 10^m + Y_R$
 - $Z1 * Z2 = (X_L * 10^m + X_R)(Y_L * 10^m + Y_R)$
 - $= X_L * Y_L * 10^{2m} + (X_L * Y_R + Y_L * X_R) 10^m + X_R Y_R$

Matrix Multiplication

- Matrix multiplication is a binary operation of multiplying two or more matrices one by one that are conformable for multiplication. For example two matrices A , B having the dimensions of $p \times q$ and $s \times t$ respectively; would be conformable for $A \times B$ multiplication only if $q=s$ and for $B \times A$ multiplication only if $t=p$.
- Matrix multiplication is associative in the sense that if A , B , and C are three matrices of order $m \times n$, $n \times p$ and $p \times q$ then the matrices $(AB)C$ and $A(BC)$ are defined as $(AB)C = A(BC)$ and the product is an $m \times q$ matrix.

Matrix Multiplication

- Matrix multiplication is not commutative. For example two matrices A and B having dimensions $m \times n$ and $n \times p$ then the matrix $AB = BA$ can't be defined. Because BA is not conformable for multiplication, even if AB are conformable for matrix multiplication.
- For three or more matrices, matrix multiplication is associative, yet the number of scalar multiplications may vary significantly depending upon how we pair the matrices and their product matrices to get the final product.

Matrix Multiplication

- Straight forward method
 - Let's suppose, we have taken two matrices A and B of size $m \times n = 2 \times 2$ and want to multiply $A \times B$ and stores the multiplication in C.

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \times B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

$$C_{ij} = \sum_{k=1}^n A_{ik} \times B_{kj}$$

$$C = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix}, \text{ where, } i \text{ and } j \text{ represents the number of rows and columns.}$$

Matrix Multiplication

- For multiplying both the matrices A and B simple algorithm will be:
- ```
for (int i = 0; i < N; i++) {
 for (int j = 0; j < N; j++) {
 C[i][j] = 0;
 for (int k = 0; k < N; k++) {
 C[i][j] += A[i][k]*B[k][j];
 }
 }
}
```

# Matrix Multiplication

- **Divide & Conquer Strategy for multiplication**
  - In divide and conquer strategy we say that if the problem is large, we break the problem into small problems called sub problems and solve those sub problems and combined solution of sub problems to get the solution of main problem.

# Matrix Multiplication

Now look at how to find the multiplication in C matrix after multiplying A and B.

$$C = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix}$$

$$C_{11} = A_{11} \times B_{11} + A_{12} \times B_{21}$$

$$C_{12} = A_{11} \times B_{12} + A_{12} \times B_{22}$$

$$C_{21} = A_{21} \times B_{11} + A_{22} \times B_{21}$$

$$C_{22} = A_{21} \times B_{12} + A_{22} \times B_{22}$$

# Matrix Multiplication

- **Strassen's Matrix Multiplication Algorithm**

Strassen's algorithm makes use of the same divide and conquer approach

- Divide the input matrices A and B into  $n/2 \times n/2$  sub-matrices, which takes  $\Theta(1)$  time.
- Now calculate the 7 sub-matrices M1-M7 by using below formulas:

$$M1 = (A11 + A22)(B11 + B22)$$

$$M2 = (A21 + A22) B11$$

$$M3 = A11 (B12 - B22) \quad M4 = A22 (B21 - B11)$$

$$M5 = (A11 + A12) B22$$

$$M6 = (A21 - A11) (B11 + B12)$$

$$M7 = (A12 - A22) (B21 + B22)$$

# Matrix Multiplication

- To get the desired sub-matrices C11, C12, C21, and C22 of the result matrix C by adding and subtracting various combinations of the  $M_i$  sub-matrices. These four sub-matrices in  $\Theta(n^2)$  time.

$$C11 = M1 + M4 - M5 + M7 \quad C12 = M3 + M5$$

$$C21 = M2 + M4$$

$$C22 = M1 - M2 + M3 + M6$$

# Important Topics

- Recurrence Relation Formulation in Divide and Conquer Technique
- Binary Search Algorithms
- Types of Sorting Algorithms
- Matrix Multiplication Algorithm`

# Summary

- In this session we have seen that Divide and Conquer approach follows a recursive approach which making a recursive call to itself until a base (or boundary) condition of a problem is not reached but with reduced problem size.
- Divide and Conquer is a top-down approach, which consists of three steps:
  - Divide: the given problem is break down into smaller parts.
  - Conquer: Solve each sub-problem by recursively calling them.
  - Combine: each sub-solution is combined to generate solution to the original problem.



*Thank  
You*



# **Design and Analysis of Algorithms**

## **Block-2 Unit-3**

### **Graph Algorithms-I**

# Topics to be Covered

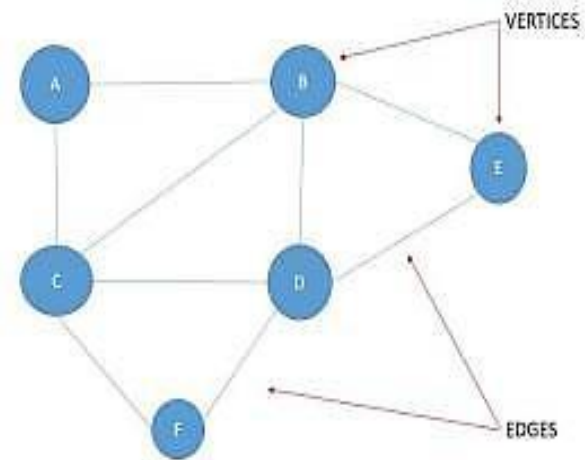
- Introduction
- Basic definition and Terminologies
- Graph Representation Schemes
- Graph Traversal Schemes
- Directed Acyclic Graph and Topological Ordering
- Strongly Connected Components
- Important Topics
- Summary

# Introduction

- Graphs are most widely used mathematical structure. It is widely used in finding shortest path routes, shortest path between every pair of vertices, in computing maximum flow problem which has applications in a large range of problems related to airlines scheduling, maximum bipartite matching and image segmentation.
- A graph can be used to model a social network which comprises millions of users or interest groups which can be represented as nodes.

# Basic Definition and Terminologies

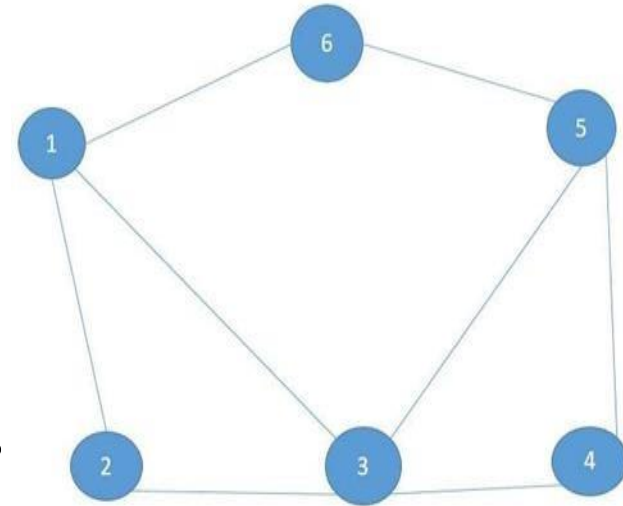
- **Graph:** A graph  $G = (V, E)$  is a data structure comprising two set of objects,  $V = \{v_1, v_2, \dots\}$  called vertices, and an another set  $E = \{e_1, e_2, \dots\}$  called the edges. In the above graph set of vertices  $V = \{A, B, C, D, E, F\}$  and set of edges  $E = \{AB, A-C, B-E, B-D, B-C, C-D, C-F, D-F, D-E, \}$ . Vertices are unordered set of nodes  $V$ .
- **Edge :** An edge is identified with an unordered pair of vertices  $(v_i, v_j)$ , where  $v_i$  and  $v_j$  are the end vertices of the edge  $e_k$ .



# Basic Definition and Terminologies

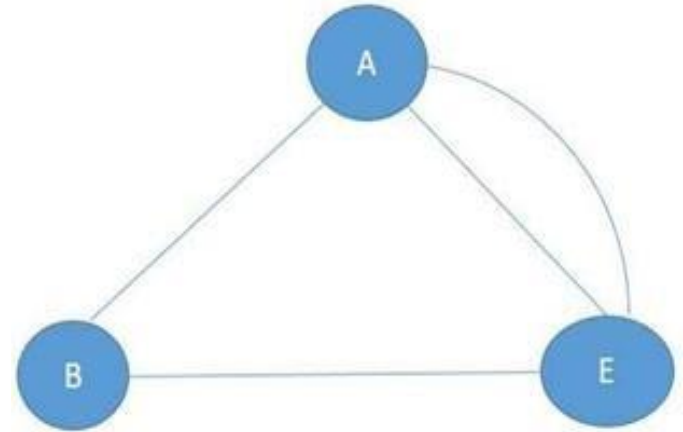
- **Graph Types:**

- A **simple graph** is a graph in which each edge is connected with two different vertices. There is no self-loop and no parallel edges in a simple graph. A simple graph.
- A simple graph with multiple edges is called multi graph.
- No vertex has a self-loop and no two vertices have more than one edge connecting them.



# Basic Definition and Terminologies

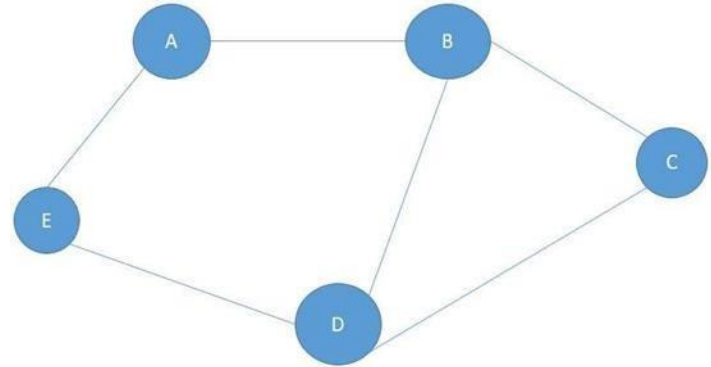
- **Graph Types:**
  - In **Multi Graph** vertex have more than one edge connecting them.



Example: Multi graph

# Basic Definition and Terminologies

- **Undirected Graph:**
  - A graph in which the edges do not have any direction and all the edges are in bi-direction.
  - In undirected graph if there is an edge from  $u$  to  $v$  then we can move from node  $u$  to node  $v$  and as well as from node  $v$  to node  $u$ .

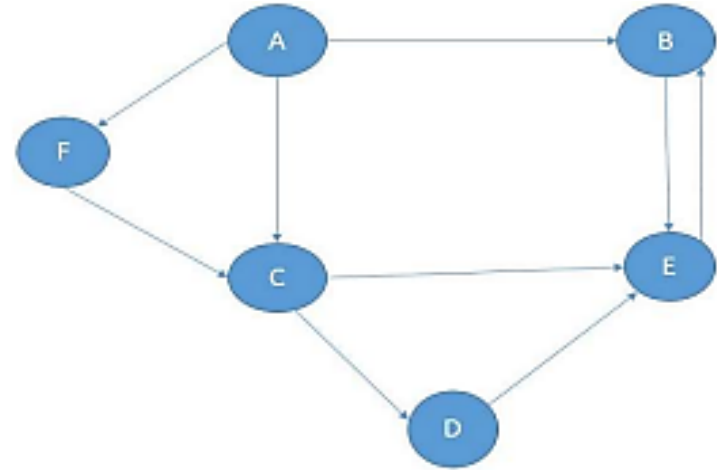


Undirected Graph



# Basic Definition and Terminologies

- **Directed Graph:** A graph in which the edges have direction. It is also called digraph.
- This is usually indicated with an arrow on the edge.
- In a directed graph if there is an edge from  $u$  to  $v$  then we can move from a node  $u$  to a node  $v$  only

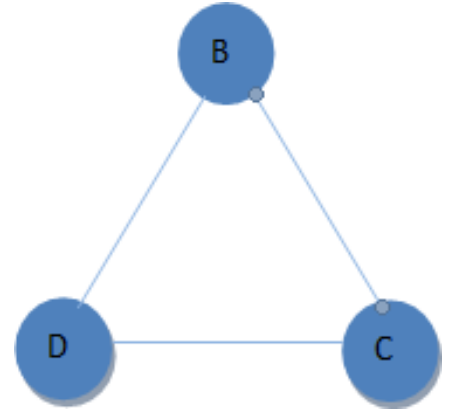


Directed graph

# Basic Definition and Terminologies

- **Subgraph:**

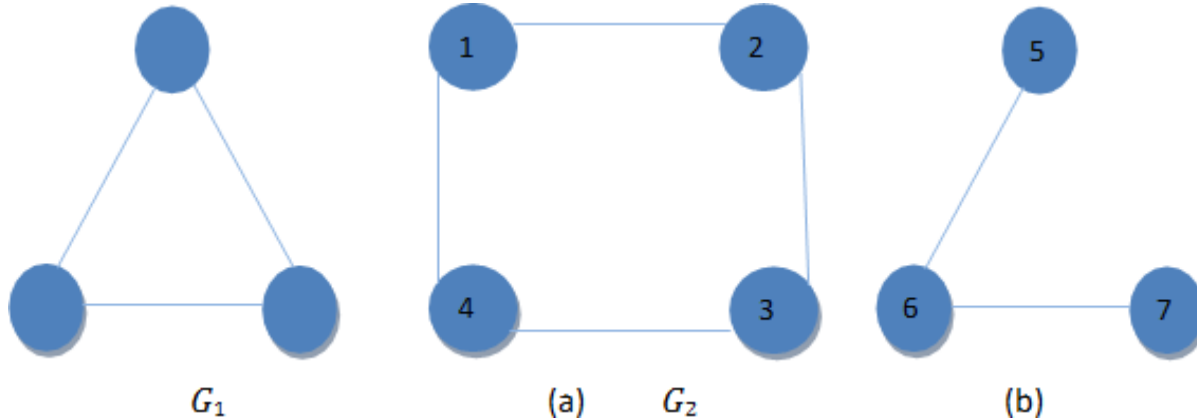
- A Subgraph of  $G$  is a graph  $G'$  such that  $V(G') \subseteq V(G)$  and  $E(G') \subseteq E(G)$  i.e., a graph whose vertices and edges are subsets of another graph. It is not necessary that a Subgraph will have all the edges of graph or all the nodes. This is a Subgraph of the graph which has the nodes  $A, B, C, D$ . In this there are  $C, B, D$  nodes.



# Basic Definition and Terminologies

- **Connected Graph:**

- An undirected graph is said to be connected if for every pair of two different vertices  $v_i$ ,  $v_j$ , there is a path between these two vertices. The graph  $G_1$  is connected whereas  $G_2$  is not connected.



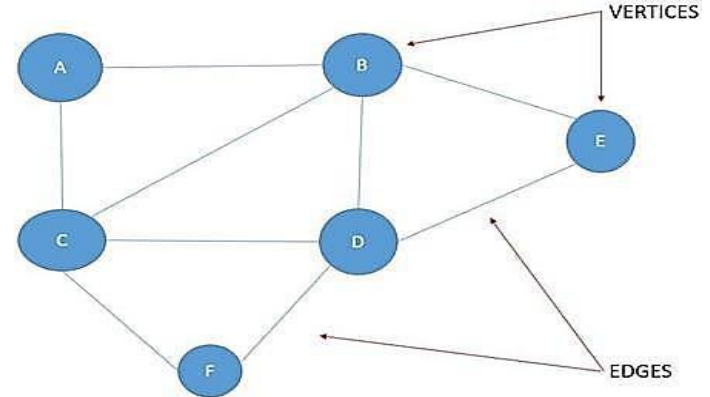
# Graph Representation

- The purpose of graph representation is to convert it to a format that can be used by algorithms running on a computer.
- In order to have efficient algorithms the logical representation of the graph plays a very critical role.
- When it comes to representations of graphs, two most standard and common computational representations are in practice such as Adjacency Matrix and Adjacency List.

# Graph Representation

- **Adjacency Matrix:**
- The adjacency matrix of a graph with  $V$  vertices is  $V \times V$  Boolean matrix with one row and one column for each of the graph's vertices. Adjacency matrix representation is typically used to represent both directed and undirected graphs.

The Boolean matrix  $A[i, j] = 1$  if there is edges between  $A_i$  &  $A_j$   
 $A[i, j] = 0$



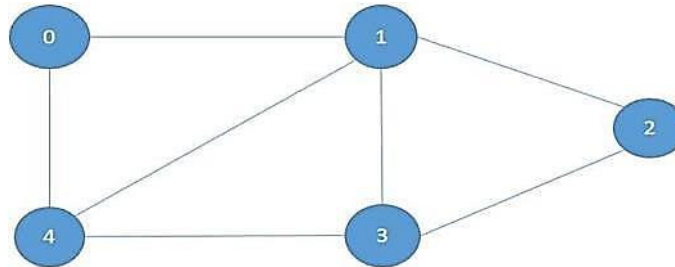
# Graph Representation

- Adjacency matrix representation of a graph given in above figure

|   | A | B | C | D | E | F |
|---|---|---|---|---|---|---|
| A | 0 | 1 | 1 | 1 | 0 | 0 |
| B | 1 | 0 | 1 | 1 | 1 | 0 |
| C | 1 | 1 | 0 | 1 | 0 | 1 |
| D | 0 | 1 | 1 | 0 | 1 | 1 |
| E | 0 | 1 | 0 | 1 | 0 | 0 |
| F | 0 | 0 | 1 | 1 | 0 | 0 |

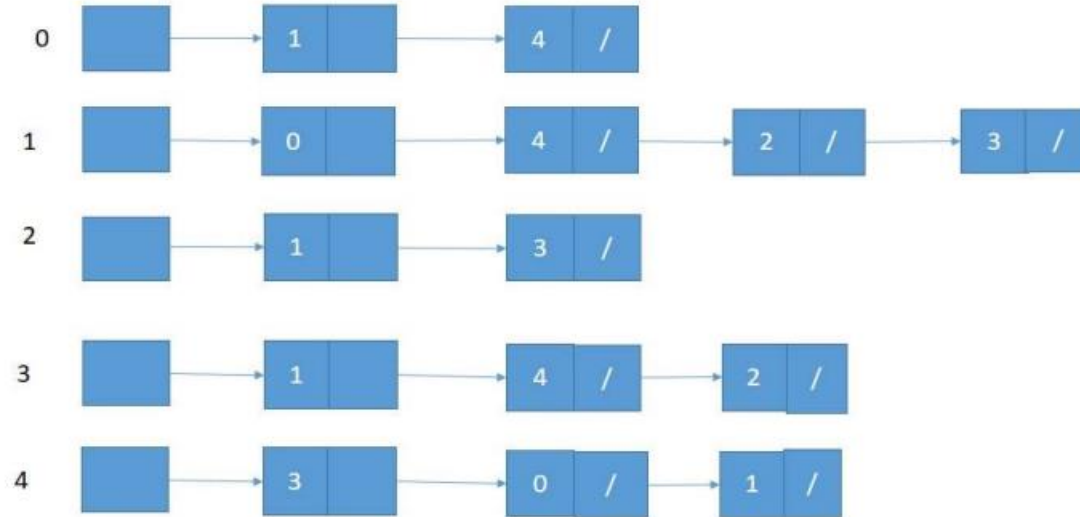
# Graph Representation

- **Adjacency List**
- Adjacency list representation is typically used to represent graphs, where the number edges  $|E|$  is much less than  $|V|^2$ . Adjacency list is represented as an array of  $|V|$  linked lists. There is one linked list for every vertex node in a graph & each node in this linked list is a reference to the other vertices which share an edge with the current vertex.



# Graph Representation

- An adjacency list of the above graph





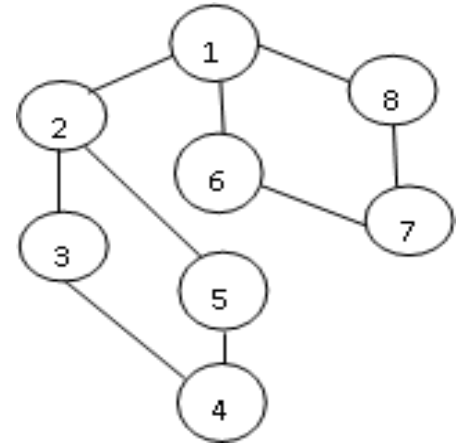
# Graph Traversal Algorithms

- Traversal algorithms are used to navigate across a given graph among all the nodes using all possible vertices. These algorithms will help us in finding the nodes, making paths that are shortest or feasible or prioritized in nature.
- There are two key graphs traversal
- Depth First Search(DFS)
  - Breadth First Search (BFS) algorithms.

# Graph Traversal Algorithms

- **Depth First Search(DFS)**

- DFS starts with any arbitrary vertex in a graph and traverses to the deepest node as far as possible and then backtracks. The algorithm proceeds as follows: it starts with selecting any arbitrary vertex as start vertex then traverses the next node  $w$  adjacent to the starting vertex  $v$ .

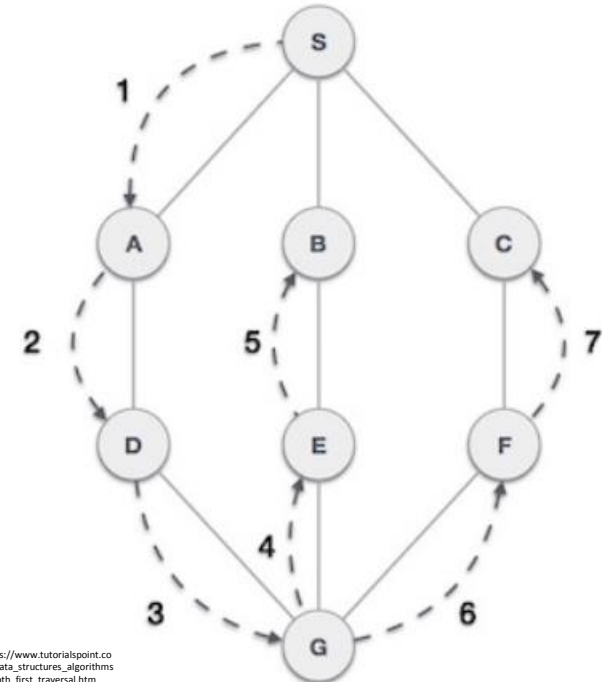


# Graph Traversal Algorithms

- **Depth First Search(DFS)** : As the name says Depth, we will return the elements of a tree or a graph in depth wise order. Let us understand from an example.

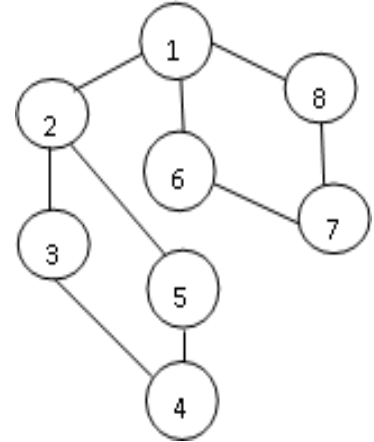
As in the example given above, DFS algorithm traverses from S to A to D to G to E to B first, then to F and lastly to C. It employs the following rules.

- **Rule 1** – Visit the adjacent unvisited vertex. Mark it as visited. Display it. Push it in a stack.
- **Rule 2** – If no adjacent vertex is found, pop up a vertex from the stack. (It will pop up all the vertices from the stack, which do not have adjacent vertices.)
- **Rule 3** – Repeat Rule 1 and Rule 2 until the stack is empty.



# Graph Traversal Algorithms

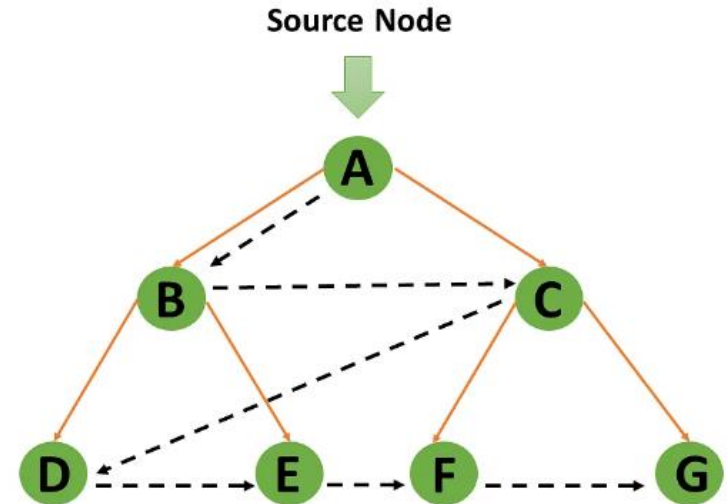
- **Breadth First Search (BFS)**
- BFS is a graph searching algorithm which start from any arbitrary vertex as a starting vertex and visits all its adjacent vertices at the first level and then moves at the second level to visit all the unvisited vertices which are adjacent to vertices at the first level vertices and so on.



# Graph Traversal Algorithms

- **Breadth First Search (BFS)** : According to the BFS, you must traverse the graph in a breadthwise direction:
  - To begin, move horizontally and visit all the current layer's nodes.
  - Continue to the next layer.

Output : A -> B -> C -> D -> E -> F -> G

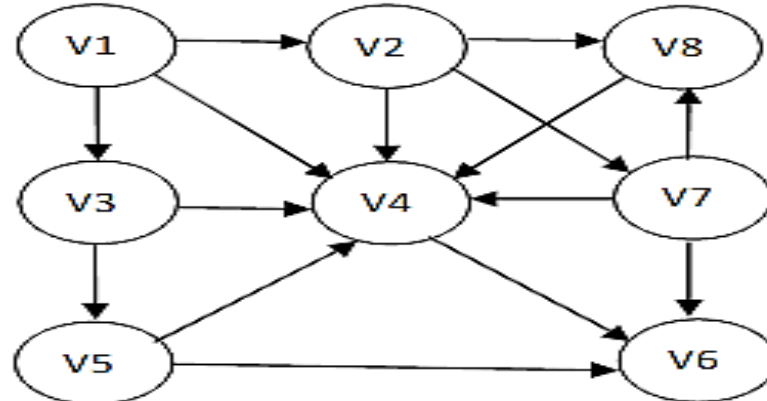


# Graph Traversal Algorithms

- **Complexities:**
  - **Time complexity:**  $O(V + E)$ , where  $O(V)$  is a total time taken to complete queue operations (insertion and deletion of vertices). Insertion and deletion of a single vertex takes  $O(1)$  unit of time .
  - Since there are  $V$  number of vertices in the graph, it will take  $O(V)$  time. The time taken to traverse each adjacency list only once is  $O(E)$ . Therefore, the total time is  $O(V + E)$ .

# Directed Acyclic Graph and Topological Ordering

- A directed graph without a cycle is called a directed acyclic graph (or a DAG (for short) which is a frequently used graph structure to represent precedence relation or dependence in a network. The following is an example of a directed acyclic task graph.



# Directed Acyclic Graph and Topological Ordering

- In this graph except vertex  $V_1$ , all other vertices are dependent upon other vertices.
- Any major task can be broken down into several subtasks. The successful completion of the task is possible only when all the subtasks are completed successfully.



# Directed Acyclic Graph and Topological Ordering

- Function topological sort( $G$ )

Input  $G = (V, E)$  //  $G$  is a DAG

{

search for a node  $V$  with *zero* in-degree (no incoming edges) and order it first in topological sorting

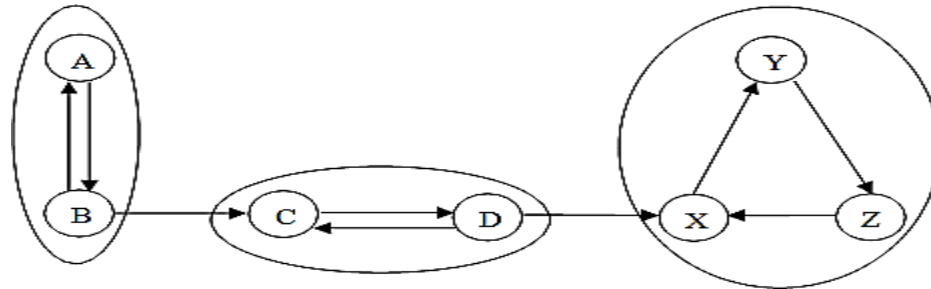
remove  $V$  from  $G$

topological sort( $G - \{V\}$ ) // recursively compute topological sorting  
append the ordering

}

# Strongly Connected Components (SCC)

- Strongly Connected Graph and Strongly Connected Components of a directed graph and then we apply an algorithm to find out whether a given directed graph is strongly connected component or not.
- A directed graph  $G = (V, E)$  is strongly connected if for every two vertices  $v_i$  and  $v_j$  there is a pair of edges from  $v_i$  to  $v_j$  and  $v_j$  to  $v_i$ . Strongly connected components is a maximal set of vertices  $M \subseteq V$  such that every pair of vertices in  $M$  are mutually reachable.



# Strongly Connected Components (SCC)

- **Pseudo code of Strongly Connected Components Strongly**
  - Perform DFS (Depth First Search) on the directed graph  $G$  and the number the vertices in the order they are visited
  - Transpose of the original  $G$  (i.e.  $G^T$ )
  - Perform DFS (Depth First Search) on  $G^T = (V, E^T)$  by starting the traversal at the highest numbered vertex
  - Print the final result.

# Important Topics

- Graph Representation Schemas
- Difference between DFS and BFS
- Directed Acyclic Graph and Topological Ordering
- Strongly Connected Components

# Summary

- In this session we have seen that a graph is a very frequently used data structure for many basic and significant algorithms.
- There are two standard approaches to represent a graph: Adjacency matrix and adjacency lists which can be used to represent both directed as well as undirected graph.
- Adjacency list provides a compact way to represent a sparse graph.
- BFS is a graph searching algorithm which start from any arbitrary vertex as a starting vertex and visits all its adjacent vertices first and then moves at the second level to visit all the unvisited vertices which are adjacent to vertices at the first level vertices and so on.

*Thank  
you*



# **Design and Analysis of Algorithms**

## **Block-3 Unit-1**

### **Graph Algorithms - II**

# Topics to be Covered

Introduction

Minimum Cost Spanning Tree (MCST)

Single Source Shortest Path

Maximum Bipartite Matching

Important Topics

Summary



# Introduction

- Graph is a non-linear data structure just like tree that consist of set of vertices and edges and it has lot of applications in computer science and in real world scenarios. Using graph algorithm easily find the shortest path, cheapest path and predicted outputs. Real Life-example of Graph:
- Maps: You can think of map as a graph where intersections of roads are vertices and connecting roads are edges.
- Social Networks: It is another example of graph structure where peoples are connected based on the friendships, or some relationships.
- Internet: You can think of internet as graph structures, where there are certain webpages and each webpages is connected through some link

# Minimum Cost Spanning Tree

- MST- A Spanning tree, weight is minimum over all spanning trees is called a minimum spanning tree or MST properties of MST are as follows:
  - A MST has  $|V|-1$  edges.
  - MST has no cycles.
  - It might not be unique.

# Minimum Cost Spanning Tree

- To find the minimum spanning tree of a graph  $G$ , two algorithms have been proposed:
  - Generic MST Algorithm
  - Kruskal's Algorithm
  - Prim's Algorithm

# Minimum Cost Spanning Tree

- **Generic MST Algorithm**

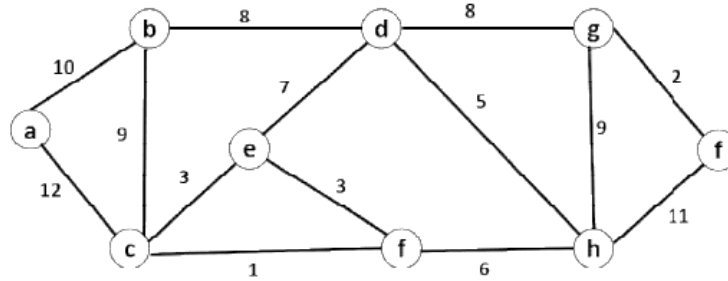
- The generic algorithm for finding MSTs maintains a subset  $A$  of the edges  $E$ . At each step, and edge  $(u, v) \in E$  is added to  $A$  if it is not already in  $A$  and its addition to the set does not violate the condition that there may not be cycles in  $A$ .

Generic-MST( $G, w$ )

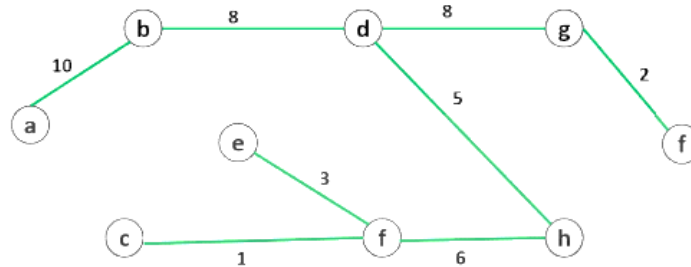
1.  $A = \{ \}$
2. while  $A$  is not a spanning tree
3. do find an edge  $(u, v)$  that is a safe for set  $A$
4.  $A = A \cup (u, v)$
5. return  $A$

# Minimum Cost Spanning Tree

Example1: Find the MST of the following graph:

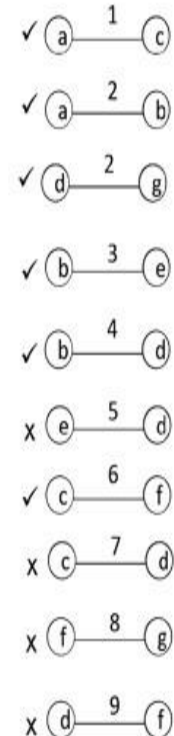
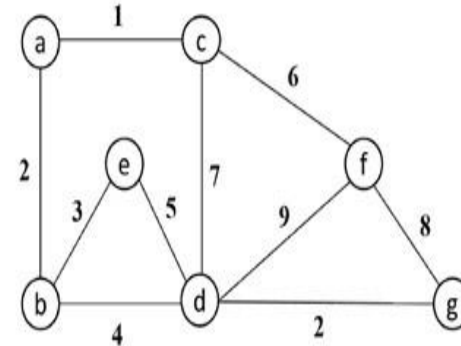


The MST of the above graph will be:



# Minimum Cost Spanning Tree

- Kruskal's Algorithm:** Kruskal's algorithm finds a minimum weighted edge (safe edge) from a graph G and add to the new sub-graph S.  
 Pseudo code of Kruskal's Algorithm:



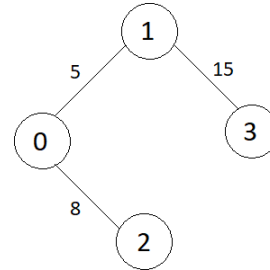
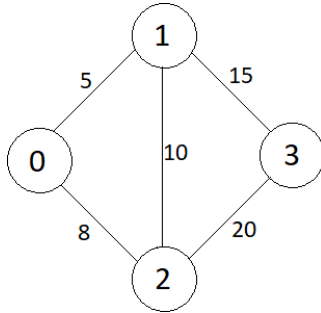
# Minimum Cost Spanning Tree

- $MCST\_Kruskal(V, E, w)$ 
  - $K \leftarrow \{ \}$
  - for each vertex  $v \in V$
  - $MAKE - SET(v)$
  - Sort the edge  $E$  of  $G$  in a non-decreasing order by weight  $w$
  - for the edge  $(u, v) \in E$ , taken from the sorted-list
  - if  $FIND - SET(u) \neq FIND - SET(v)$
  - then  $K \leftarrow K \cup \{(u, v)\}$
  - $UNION(u, v)$
  - return  $K$

# Graph Traversal Algorithms

- **PRIM's Algorithm**

- Prim's algorithm based on the greedy algorithm to find the minimum cost spanning tree & it finds safe edge .



- Minimum cost={0,5,8,15}



# Graph Traversal Algorithms

- **Working strategy of Prim's algorithm.**
  - We begin with some vertex  $v$  in a given graph  $G(V, E)$  defines the initial set of vertex in  $K$ .
  - Next choose a minimum weight edge  $(u, v) \in E$  in the graph that have one end vertex  $u$  in the set  $K$  and vertex  $v$  outside of the set  $K$ .
  - Then vertex  $v$  added in the set  $K$ .
  - Repeat this process until you get the  $|V| - 1$  edges in the spanning tree.

# Graph Traversal Algorithms

- Pseudo code of Prim's Algorithm:

MCST\_Prim's(  $V, E, w, r$  )

1.  $SE \leftarrow \{\}$
2.  $SV \leftarrow \{r\}$
3. while  $E \neq \emptyset$  & size  $|SE| \neq |V| - 1$
4.     Select minimum weight edge  $(u, v)$  from  $G$
5.     If  $\sim(u \in SV \text{ and } v \notin SV)$
6.         break;
7.      $E = E - \{(u, v)\}$
8.      $SE = SE \cup \{(u, v)\}$
9.      $SV = SV \cup \{u\}$
10.    if  $(|SE| == |V| - 1)$
11.      $SE$  is a minimum spanning tree that contains  $|V| - 1$  vertices
12.    else
13.     Graph is disconnected

# Single Source Shortest Path

- In real world life graph can be used to represent the cities and their connections between each city.
- Vertices represents the cities and edges representing roads that connects these vertices.
- The edges can have weights which may be the miles from one city to other city. Suppose a person wants to drive from a city **P** to city **Q**.
- He may be interested to know the following queries:
  - Is there any path exist from **P** to **Q**?
  - If there are multiple paths from **P** to **Q**, then which is the shortest path?

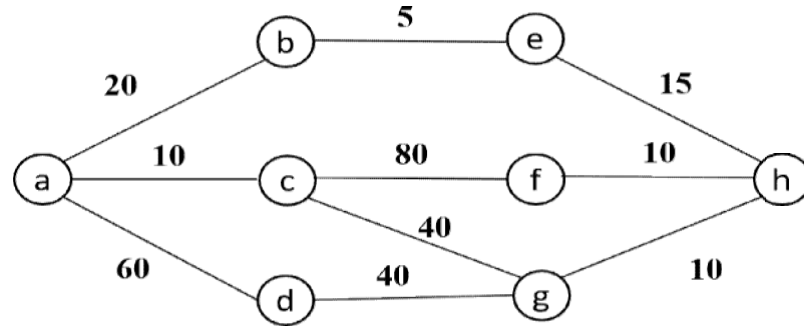
# Single Source Shortest Path

- There are two algorithm to solve the single-source-shortest path problem.
  - Dijkstra's Algorithm
  - Bellman Ford Algorithm

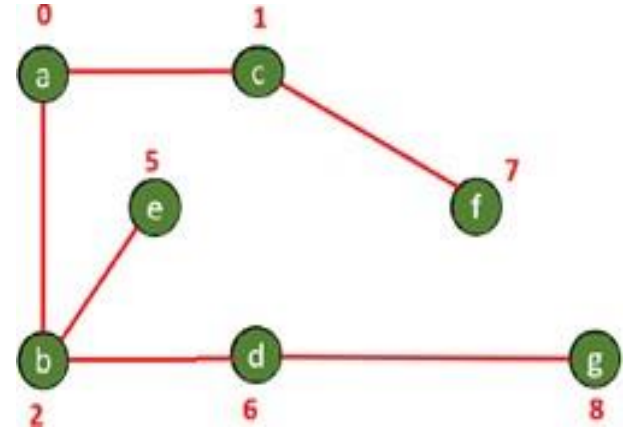
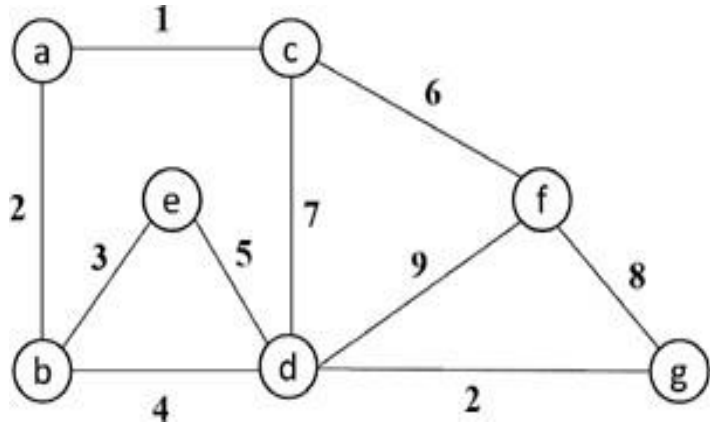
# Single Source Shortest Path

- **Dijkstra's Algorithm**

- Dijkstra's algorithm solves the single-source shortest path problem when all edges have non-negative weights.
- It is a greedy algorithm's similar to Prim's algorithm and always choose the path that are optimal right now not for future consequences.

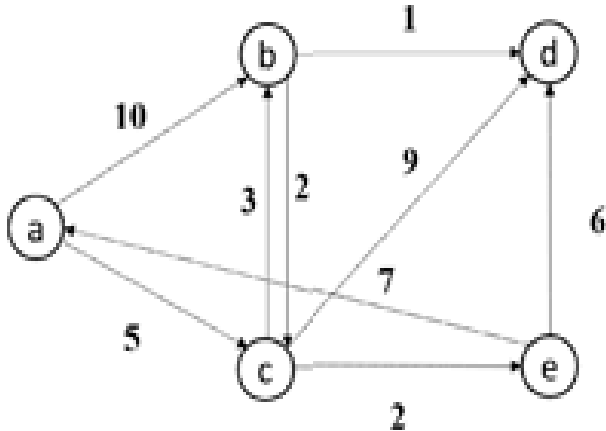


# Single Source Shortest Path



# Single Source Shortest Path

- Apply Dijkstra's algorithm on directed graph from source vertex *a*



| Selected vertex | a | b        | c        | d        | e        |
|-----------------|---|----------|----------|----------|----------|
| a               | 0 | $\infty$ | $\infty$ | $\infty$ | $\infty$ |
| c               |   | 10       | 5        | $\infty$ | $\infty$ |
| e               |   | 8        |          | 14       | 7        |
| b               |   | 8        |          | 13       |          |
| d               |   |          |          | 9        |          |

# Single Source Shortest Path

- **Bellman-Ford Algorithm**

The Bellman-Ford algorithm solves the single-source shortest-paths problem in case where edge weights may be negative or there is a negative edge weight cycle in the graph.



# Single Source Shortest Path

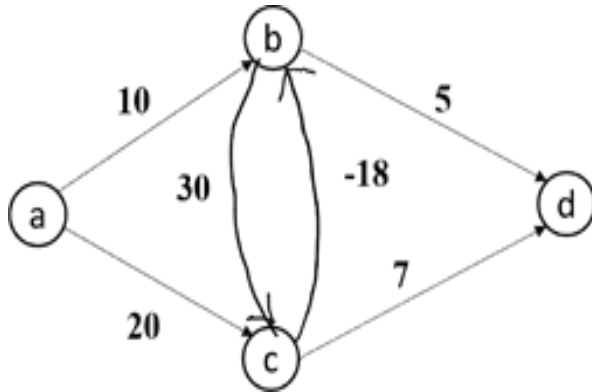
- **Pseudo-code of Bellman-Ford Algorithm**

*BELLMAN – FORD*( $G, V, E, w, s$ )

- Initialize  $\text{dist}[s] \leftarrow 0$
- for each vertex  $v \in V - s$
- $\text{dist}[V] \leftarrow \infty$
- for each vertex  $i=1$  to  $V - 1$
- for each edge  $(u, v)$  in  $E[G]$
- $\text{RELAX}(u, v, w)$
- for each edge  $\text{edge}(u, v)$  in  $E[G]$
- if  $\text{dist}[u] + w(u, v) < \text{dist}[v]$
- return FALSE
- return TRUE

# Single Source Shortest Path

- Apply Bellman-Ford Algorithm on a following graph



| a | b        | c        | d        |
|---|----------|----------|----------|
| 0 | $\infty$ | $\infty$ | $\infty$ |
| 0 | 10       | 20       | $\infty$ |
| 0 | 10       | 20       | 15       |
| 0 | 2        | 20       | 15       |

| a | b        | c        | d        |
|---|----------|----------|----------|
| 0 | $\infty$ | $\infty$ | $\infty$ |
| 0 | 10       | 20       | $\infty$ |
| 0 | 10       | 20       | 15       |
| 0 | 2        | 20       | 15       |
| 0 | 2        | 20       | 7        |

# Single Source Shortest Path

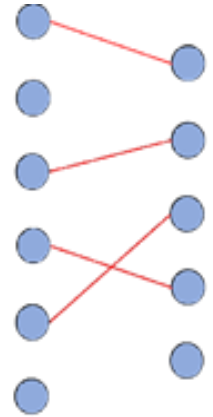
- **Time Complexity of Bellman-Ford Algorithm:**
  - The initialization in line 1 takes  $O(V)$  time. For loop of line 2 takes  $O(V)$  time. For loop of lines 3-4, it takes  $O(E)$  time and for-loop of line 5-7, it takes  $O(E)$ . Therefore, Bellman-ford runs in  **$O(VE)$** .

# Maximum Bipartite Matching

- Maximum bipartite matching problem is a subset of matching problem specific for bipartite graphs. A matching of maximum size (i.e. maximum number of edges) is known to be maximum matching if the addition of any edge makes it no longer a matching.



L R  
(a) A bipartite graph  $G = (V, E)$  with  $V = L \cup R$



L R  
(b) A maximum bipartite graph matching

# Important Topics

- Representation of graph
- Minimum cost spanning tree
- Dijkstra's Algorithm
- Bellman-Ford Algorithm
- Maximum Bipartite Matching

# Summary

- In this session we have seen that :-
- Graph is a non-linear data structure just like tree that consist of set of vertices and edges and it has lot of applications in computer science and in real world scenarios.
- A connected sub graph  $S$  of Graph  $G(V, E)$  is said to be spanning tree if and only if it contains all the vertices of the graph  $G$  and have minimum total weight of the edges of  $G$ .
- kruskal's algorithm prim's algorithm also based on the greedy algorithm to find the minimum cost spanning tree.
- Dijkstra's algorithm solves the single-source shortest path problem when all edges have non-negative weights

*Thank  
You*



# **Design and Analysis of Algorithms**

## **Block-3 Unit-2**

### **Dynamic Programming Technique**



# Topics to be Covered

- Introduction
- Principal of Optimality
- Matrix Multiplication
- Matrix Chain Multiplication
- Optimal Binary Search Tree
- Binomial Coefficient Computation
- All Pair shortest Path
- Important Topics
- Summary

# Introduction

- Dynamic programming is an optimization technique. Optimization problems are those which required either minimum result or maximum result .
- Dynamic programming granted to find the optimal solution of any problem if their solution exist .
- Dynamic programming is useful in solving the problems which can be divided into similar sub problems. If each sub problem is different in nature, Dynamic programming does not help in reducing the time complexity

# Principal of Optimality

- Dynamic programming follows the principal of optimality. If a problem have an optimal structure, then definitely it has principal of optimality. A problem has optimal sub structure if an optimal solution can be constructed efficiently from optimal solution of its sub-problems.
- Principal of optimality shows that a problem can be solved by taking a sequence of decision to solve the optimization problem. In dynamic programming in every stage we takes a decision. The Principle of Optimality states that components of a globally optimum solution must themselves be optimal.

# Principal of Optimality

- The shortest path problem satisfies the Principle of Optimality.
- This is because if  $a, x_1, x_2, \dots, x_n$  is a shortest path from node  $a$  to node  $b$  in a graph, then the portion of  $x_i$  to  $x_j$  on that path is a shortest path from  $x_i$  to  $x_j$ .
- The longest path problem, on the other hand, does not satisfy the Principle of Optimality. For example the undirected graph of nodes  $a, b, c, d$ , and  $e$  and edges  $(a, b), (b, c), (c, d), (d, e)$  and  $(e, a)$ . That is,  $G$  is a ring. The longest (non cyclic) path from  $a$  to  $d$  is  $a, b, c, d$ . The sub-path from  $b$  to  $c$  on that path is simply the edge  $b, c$ . But that is not the longest path from  $b$  to  $c$ . Rather  $b, a, e, d, c$  is the longest path.

# Matrix Multiplication

- Matrix multiplication is a binary operation of multiplying two or more matrices one by one that are conformable for multiplication. For example two matrices A, B having the dimensions of  $p \times q$  and  $s \times t$  respectively; would be conformable for  $A \times B$  multiplication only if  $q=s$  and for  $B \times A$  multiplication only if  $t=p$ .
- Matrix multiplication is associative in the sense that if A, B, and C are three matrices of order  $m \times n$ ,  $n \times p$  and  $p \times q$  then the matrices  $(AB)C$  and  $A(BC)$  are defined as  $(AB)C = A(BC)$  and the product is an  $m \times q$  matrix.

# Matrix Multiplication

- Matrix multiplication is not commutative. For example two matrices A and B having dimensions  $m \times n$  and  $n \times p$  then the matrix  $AB = BA$  can't be defined. Because BA are not conformable for multiplication even if AB are conformable for matrix multiplication.
- For 3 or more matrices, matrix multiplication is associative, yet the number of scalar multiplications may very significantly depending upon how we pair the matrices and their product matrices to get the final product.

# Matrix Chain Multiplication

- Given a sequence of matrices that are conformable for multiplication in that sequence, the problem of matrix-chain-multiplication is the process of selecting the optimal pair of matrices in every step in such a way that the overall cost of multiplication would be minimal.
- If there are total  $N$  matrices in the sequence then the total number of different ways of selecting matrix-pairs for multiplication will be  $2^n C_n / (n+1)$ .

# Matrix Chain Multiplication

- **Step 1: The structure of an optimal Parenthesization:**
  - An optimal parenthesization of  $A_1 \dots A_n$  must break the product into two expressions, each of which is parenthesized or is a single array
  - Assume the break occurs at position  $k$ .
  - In the optimal solution, the solution to the product  $A_1 \dots A_k$  must be optimal
    - Otherwise, we could improve  $A_1 \dots A_n$  by improving  $A_1 \dots A_k$ .
    - But the solution to  $A_1 \dots A_n$  is known to be optimal
    - This is a contradiction
    - Thus the solution to  $A_1 \dots A_n$  is known to be optimal



# Matrix Chain Multiplication

- This problem exhibits the Principle of Optimality:
  - The optimal solution to product  $A_1 \dots A_n$  contains the optimal solution to two sub products
- Thus we can use Dynamic Programming
  - Consider a recursive solution
  - Then improve it's performance with memorization or by rewriting bottom up

# Matrix Chain Multiplication

- **Step 2: A recursive solution**

- For the matrix-chain multiplication problem, we pick as our sub problems the problems of determining the minimum cost of parenthesizing  $A_i A_{i+1} \dots A_j$  for  $1 \leq i \leq j \leq n$ .
- Let  $m[i, j]$  be the minimum number of scalar multiplication needed to compute the matrix  $A_i \dots j$ ; for the full problem, the lowest cost way to compute
- $A_1 \dots n$  would thus be  $m[1, n]$ .
- $m[i, i] = 0$ , when  $i=j$ , problem is trivial. Chain consist of just one matrix

# Matrix Chain Multiplication

- $A_{i\dots i} = A_i$ , so that no scalar multiplications are necessary to compute the product.
- The optimal solution of  $A_i \times A_j$  must break at some point,  $k$ , with  $\leq k < j$ .
- Each matrix  $A_i$  is  $p_{i-1} \times p_i$  and computing the matrix multiplication  $A_{i\dots k} A_{k+1\dots j}$  takes  $p_{i-1}p_kp_j$  scalar multiplication.
- Thus,  $m[i, j] = m[i, k] + m[k + 1, j] + p_{i-1}p_kp_j$  Equation 1

# Matrix Chain Multiplication

- **Step 3 :Computing the Optimal Costs**

MATRIX-CHAIN-ORDER (P)

1.  $n \leftarrow \text{length}[p] - 1$
2. let  $m[1..n, 1..n]$  and  $s[1..n-1, 2..n]$  be new tables
3. for  $i \leftarrow 1$  to  $n$
4.     do  $m[i, i] \leftarrow 0$
5.   For  $l \leftarrow 2$  to  $n$
6.     do for  $i \leftarrow 1$  to  $n-l+1$
7.       do  $j \leftarrow i+l-1$

# Matrix Chain Multiplication

```
8. $m[i, j] \leftarrow \infty$
9. for $k \leftarrow i$ to $j-1$
10. do $q \leftarrow m[i, k] + m[k + 1, j] + p_{i-1}p_kp_j$
11. if $q < m[i, j]$
12. then $m[i, j] \leftarrow q$
13. $s[i, j] \leftarrow k$
14. return m and s
```

# Matrix Chain Multiplication

- **Step 4 :Constructing an Optimal Solution**
  - The MATRIX-CHAIN-ORDER determines the optimal number of scalar multiplication needed to compute a matrix-chain product. To obtain the optimal solution of the matrix multiplication, we call **PRINT\_OPTIMAL\_PARES(s,1,n)** to print an optimal parenthesization of  $A_1A_2, \dots, A_n$ . Each entry  $s[i, j]$  records a value of  $k$  such that an optimal parenthesization of  $A_1A_2, \dots, A_j$  splits the product between  $A_k$  and  $A_{k+1}$ .

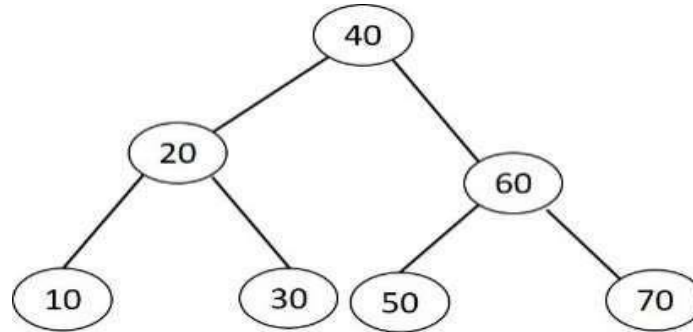
# Matrix Chain Multiplication

PRINT\_OPTIMAL\_PARENS(s,i,j)

1. If i==j
2. Then print "A"i
3. else print "("
4. PRINT\_OPTIMAL\_PARENS(s, i, s[i,j])
5. PRINT\_OPTIMAL\_PARENS(s, s[i,j]+1, j)
6. print ")"

# Optimal Binary Search Tree

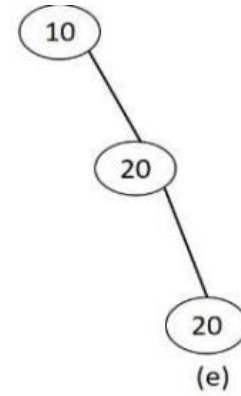
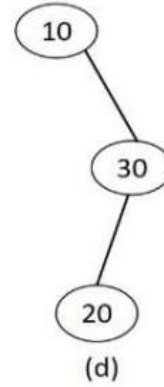
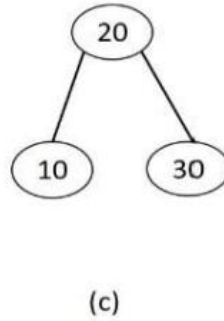
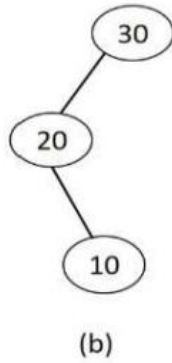
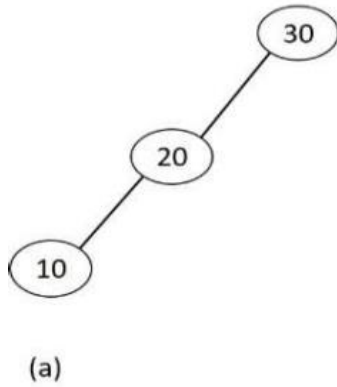
- Binary Search Tree: A binary tree is binary search tree such that all the key smaller than root are on left side and all the keys greater than root or on right side.
- No of comparison required to search any element in a binary search tree is depend upon the number of levels in a binary search tree. We are searching for the key that are already present in the binary search tree is successful search.





# Optimal Binary Search Tree

- Let's take an example: Keys: 10, 20, 30 How many binary search tree possible.  
 $T(n) = \frac{2n!}{n+1}$
- So there are 3 keys number of binary search tree possible = 5. Let's draw five possible binary search tree shown in below figure (a)-(e).



# Optimal Binary Search Tree

- **Dynamic Programming Approach:**

- Optimal Substructure: if an optimal binary search tree  $T$  has a sub tree  $T'$  containing keys  $k_i, \dots, k_j$ , then this sub tree  $T'$  must be optimal as well for the sub problem with keys  $k_i, \dots, k_j$  and dummy keys  $d_{i-1}, \dots, d_j$ .
- Algorithm for finding optimal tree for sorted, distinct keys  $k_i, \dots, k_j$ :
  - For each possible root  $k_r$  for  $i \leq r \leq j$
  - Make optimal sub tree for  $k_i, \dots, k_{r-1}$ .
  - Make optimal sub tree for  $k_{r+1}, \dots, k_j$ .
  - Select root that gives best total tree

# Optimal Binary Search Tree

- Recursive solution: We pick our sub problem domain as finding an optimal binary search tree containing the keys  $k_i, \dots, k_j$ , where  $i \geq 1, j \leq n$ , and
- $j \geq i - 1$ . Let us define  $e[i, j]$  as the expected cost of searching an optimal binary search tree containing the keys  $k_i, \dots, k_j$ .  
Ultimately, we wish to compute  $e[1, n]$ .

# Optimal Binary Search Tree

## •Computing the Optimal Cost:

OPTIMAL\_BST( $p, q, n$ )

1. let  $e[1..n + 1, 0..n]$ ,  $w[1..n + 1, 0..n]$  and  $root[1..n, 1..n]$  be new tables.
2. for  $i=1$  to  $n+1$
3.  $e[i, i - 1] = q_{i-1}$
4.  $w[i, i - 1] = q_{i-1}$
5. for  $l=1$  to  $n$
6.     for  $j=1$  to  $n-l+1$
7.          $J=i+l-1$

# Optimal Binary Search Tree

8.  $e[i, j] = \infty$
9.  $w[i, j] = w[i, j - 1] + p_j + q_i$
10.       for  $r=i$  to  $j$
11.              $t = e[i, r-1] + e[r+1, j] + w[i, j]$
12.             if  $t < e[i, j]$
13.  $e[i, j] = t$
14.  $root[i, j] = r$
15. return  $e$  and  $root$ .

# Binomial Coefficient Computation

- Computing binomial coefficients is non optimization problem but can be solved using dynamic programming. Binomial Coefficient is the coefficient in the Binomial Theorem which is an arithmetic expansion. It is denoted as  $c(n, k)$  which is equal to  $n! / (k! \times (n-k)!)$  where ! denotes factorial.
- This follows a recursive relation using which we will calculate the  $n$  binomial coefficient in linear time  $O(n \times k)$  using Dynamic Programming

# Binomial Coefficient Computation

- **What is Binomial Theorem?**

Binomial is also called as Binomial Expansion describe the powers in algebraic equations. Binomial Theorem helps us to find the expanded polynomial without multiplying the bunch of binomials at a time. The expanded polynomial will always contain one more than the power you are expanding.

$$(x + a)^n = \sum_{k=0}^n \binom{n}{k} x^k a^{n-k}$$

# All Pair Shortest Path

- **Floyd Warshall Algorithm**

- Floyd Warshall Algorithm uses the Dynamic Programming (DP) methodology. Unlike Greedy algorithms which always looks for local optimization, DP strives for global optimization that means DP does not relies on immediate best result. It works on the concept of recursion i.e. dividing a bigger problem into similar type of sub problems and solving them recursively.
- Floyd Warshall Algorithm works on directed weighted graph including negative edge weight. Although it does not work on graph having negative edge weight cycle.



# All Pair Shortest Path

- **Working Strategy for FWA**

- Initial Distance matrix  $D^0$  of order  $n \times n$  consisting direct edge weight is taken as the base distance matrix.  $i, j$
- Another distance matrix  $D^1$  of shortest path  $d^{(1)}$  including one (1) intermediate vertex is calculated where  $i, j \in V$ .
- The process of calculating distance matrices  $D^2, D^3 \dots D^n$  by including other intermediate vertices continues until all vertices of the graph is taken.
- The last distance matrix  $D^n$  which includes all vertices of a matrix as intermediate vertices gives the final result.

# All Pair Shortest Path

- **Pseudo-code for FWA**

- **N** is the number of vertices in graph **G (V, E)**.
- **$D^k$**  is the distance matrix of order  **$n \times n$**  consisting matrix element  **$d^{(k)}$**  as the weight of shortest path having intermediate vertices from  **$1, 2 \dots k \in V$**
- **M** is the initial matrix of direct edge weight. If there is no direct edge between two vertices, it will be considered as  $\infty$ .

# All Pair Shortest Path

FWA(M)

1.  $D^0 = M$
2. For (k=1 to n)
3. For (i=1 to n)
4.                      For (j=1 to n)
5.  $d_{\underline{i}\underline{j}}^{(k)} = \min \{d_{\underline{i}\underline{j}}^{(k-1)}, d_{\underline{i}\underline{k}}^{(k-1)} + d_{\underline{k}\underline{j}}^{(k-1)}\}$
6. Return  $D^n$

# All Pair Shortest Path

- **When FWA gives the best result**
  - Floyd Warshall Algorithm for finding all pair shortest path at one go or one can also use Dijkstra's or Bellman Ford algorithm for each vertex?.
  - Now we will find out the time complexities of running Dijkstra's and Bellman Ford algorithm for finding all pair shortest path to see which gives the best result.

# All Pair Shortest Path

- **Using Dijkstra's Algorithm** - The time complexity of running Dijkstra's Algorithm for single source shortest path problem on graph  $G(V,E)$  is  $O(E+V\log V)$  using Fibonacci heap.
- For  $G$  being complete graph, running Dijkstra's Algorithm on each vertex will result in time complexity of  $O(V^3)$ .
- For graph other than complete, it shall be  $O(n*n \log n)$ , less than FWA but since Dijkstra's Algorithm does not work with negative edge weight for single source shortest path problem it will also not work for all pair shortest path problem given negative edge weight.

# All Pair Shortest Path

- **Using Bellman Ford Algorithm** – The time complexity of running Bellman Ford algorithm for single source shortest path problem on Graph  $G(V,E)$  is  $O(VE)$ . If  $G$  is complete graph then this complexity turns out to be  $O(V*V^2)$  i.e.  $O(V^3)$ . Therefore, the time complexity of running Bellman Ford Algorithm on each vertex of graph  $G$  shall be  $O(V^4)$ .

# All Pair Shortest Path

- From the two points it can be concluded that FWA is the best choice for all pair shortest path problem when the graph is dense whereas Dijkstra's Algorithm is suitable when the graph is sparse and no negative edge weight exist. For graph having negative edge weight cycle the only choice among the three is Bellman Ford Algorithm.

# Important Topics

- Principal of Optimality
- Matrix Multiplication
- Matrix Chain Multiplication
- Binary Search Tree
- Binomial Coefficient Computation
- Floyd Warshall Algorithm
- Working Strategy for FWA
- Pseudo-code for FWA



# Summary

- In the session of Dynamic Programming we have seen it is a technique for solving optimization Problems, using bottom-up approach.
- The underlying idea of dynamic programming is to avoid calculating the same thing twice, usually by keeping a table of known results that fills up as substances of the problem under consideration are solved.
- In order that Dynamic Programming technique is applicable in solving an optimization problem, it is necessary that the principle of optimality is applicable to the problem domain.
- The principle of optimality states that for an optimal sequence of decisions or choices, each subsequence of decisions/choices must also be optimal

*Thank  
You*



# **Design and Analysis of Algorithms**

## **Block-3 Unit-3**

### **String Matching Techniques**

# Topics to be Covered

- Introduction
- Naive or Brute Force Algorithm
- Rabin Karp Algorithm
- Knuth-Morris-Pratt Algorithm
- Important Topics
- Summary

# Introduction

- String matching is an important problem in computer science in which a sub-string ( also called a pattern) is searched in a larger string or a text (e.g., a sentence, a paragraph of a book) and returns the index of a starting character of a substring.
- Approximate String Matching Algorithms is known as Fuzzy String Searching , search for substrings of the input string.
- The purpose of the string matching algorithm is to search for a location of a smaller text pattern in a paragraph or a book .

# The Naïve or Brute Force Algorithm

- String matching is an important problem in computer science in which a sub-string (also called a pattern) is searched in a larger string or a text (e.g., a sentence, a paragraph of a book) and returns the index of a starting character of a substring.
- The Naïve search algorithm compares the pattern string to the text, one character at a time. This process continues until there is mismatch of characters between the pattern string and the text. Pseudo-Code -

do

```
if (pattern string character == text character) {
 compare next character of the pattern string to next character of text character;
}
else{
 shift the pattern string to the next character of the text; while(entire
 pattern string matched or end of the text)
```

# The Rabin Karp Algorithm

- The central idea in Rabin Karp algorithm is computation of hash function to speed up the pattern matching. The algorithm calculates hash values for
  - ( i) pattern string of m- characters
  - (ii) m-character substring of a text .
- Advantage is that there is only one comparison per text substring .

# The Rabin Karp Algorithm

- **Pseudo-code of Robin –Karp Algorithm**

m- length of a pattern substring //Input

P\_hash – Hash value of a pattern string //Input

T\_hash – Hash value of a first m character of a text substring

//Input

do

if( P\_hash == T\_hash) {brute force comparison of the pattern string and the first m-character text substring ; }

else{

T\_hash = hash value of the next m-character of the text substring after one character shift;}

while( match of the pattern string or end of the text)



# The Rabin Karp Algorithm

- **Best Case –  $O(n)$**  where  $n$  is a length of a text. If sufficiently large base number or a large prime number is used for computing hash value , there will be no spurious hits and the hashed values would be distinct for both a pattern string and a text substring. In such a case , the searching would take  $O(n)$  time.
- **Worst Case =  $O(mn)$**  where  $m$  is a length of a pattern string and  $n$  is a length of a text string.. This may happen if there are spurious hits because of use a small base number/prime number in hash calculation of a pattern and a text string.

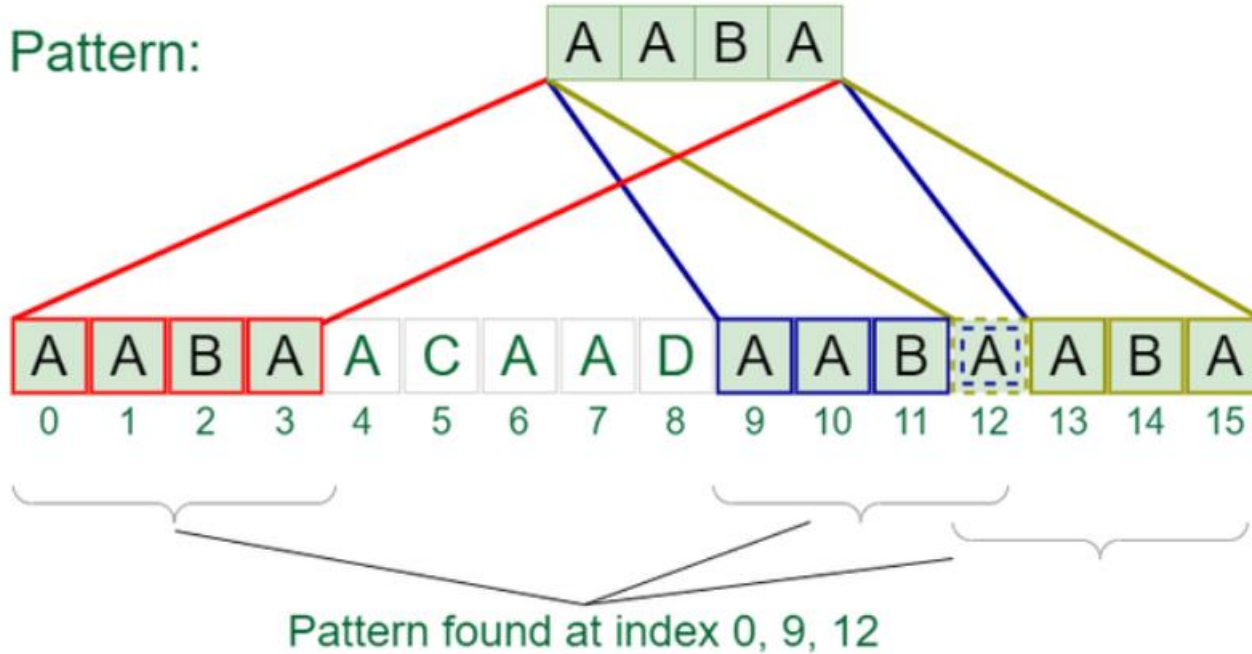
# Knuth Morris Pratt Algorithm

- This is linear time string matching algorithm. The complexity is  $O(m + n)$  where  $m$  and  $n$  are the length of a pattern string and a text string respectively.
- This happens because the KPS algorithm avoids frequent backtracking in the text string as it is done in the naïve algorithm.
- The key idea in KMP algorithm is to build a LPS( largest prefix as suffix) array to determine from which point in the pattern string to restart comparing for pattern matching in a text in case there is a mismatch of a character without moving the text pointer backward.

# Knuth Morris Pratt Algorithm

Text: A A B A A C A A D A A B A A B A

Pattern:



# Important Topics

- Naive or Brute Force Algorithm
- Rabin Karp Algorithm
- Knuth-Morris-Pratt Algorithm

# Summary

- In this session we have seen that :-
- String matching is a problem for a pattern string into a larger text and return the location where the pattern has occurred in the Text.
- The Rabin Karp algorithm is based on computing the hash values of an  $m$ -character pattern string and an  $m$ -character text substring.
- The KMP is linear time algorithm. The algorithm forbids moving a text pointer backward.

*Thank  
You*



# **Design and Analysis of Algorithm**

## **Block-4 Unit-1**

### **Introduction to Complexity Classes**

# Topics to be Covered

- Introduction
- Some Preliminaries to P and NP Class of Problems
- Introduction to P and NP, and NP- Complete Problems
- The CNF Satisfiability Problem
- Important Topics
- Summary



# Introduction

- The NP complete problem which is a basis of all other NP complete problems.
- Problems from graph theory and combinatory can be formulated as a language recognition problem, for example, pattern matching problems which can be solved by building automata.

# Some Preliminaries to P and NP Complexity Classes

- **Tractable Vs. Intractable Problems**
  - The general view is that the problems are hard or intractable if they can be solved only in exponential time or factorial time. The opposite view is that the problems having polynomial time solutions are tractable or easy problems.
  - Although the exponential time function such as  $2^n$  grows more rapidly than any polynomial time algorithms for an input size  $n$ , but for small values of  $n$ , intractable problems for with exponential time bounded complexity can be more efficient than polynomial time complexity. But in the asymptotic analysis of algorithm complexity, we always assume that the size of  $n$  is very large .

# Some Preliminaries to P and NP Complexity Classes

- **Tractable Vs. Intractable Problems**
  - It is to be kept in mind that intractability is a characteristic of a problem. It is not related to any problem solving technique. To explain this concept, let us take an example of a chained matrix multiplication problem which was examined earlier.
  - The brute force approach to the solution of this problem is intractable but it can be solved in polynomial time through dynamic programming technique.

# Some Preliminaries to P and NP Complexity Classes

- **Optimization Problems Vs. Decision Problems**
  - An Optimization problem is one in which we are given a set of input values, which are required to be either maximized or minimized w. r. t. some constraints or conditions.
  - There is a corresponding decision problem to each optimization problem. Unlike an optimization problem, a decision problem outputs a simple “yes” or “no” answer.

# Some Preliminaries to P and NP Complexity Classes

- **Deterministic Vs. Nondeterministic Algorithms**
  - A deterministic algorithm will produce the same output on an input going through the same sequence of steps,
  - A non-deterministic algorithm behaves in a completely different way.
  - There may be multiple next steps possible after any given step and the algorithm is allowed to choose any of them in an arbitrary manner.
  - A non-deterministic algorithm may proceed through different sequences of steps on different runs, and may even produce different outputs.

# Introduction to P, NP, NP Hard & NP-Complete Problems

- The theory of NP-completeness identifies a large class of problems which cannot be solved in polynomial time. Categories the problems into three classes namely P (Polynomial), NP (Non-deterministic Polynomial), and NP complete.

# Introduction to P, NP, NP Hard & NP-Complete Problems

- **P Class**

- An algorithm solves a problem in polynomial time if its worst -case time complexity belongs to  $O(p(n))$  where  $n$  is a size of a problem and  $p(n)$  is polynomial of the problem's input size  $n$ .
- Problems can be classified as tractable and intractable problems. Problems with polynomial time solutions are called tractable, problems which do not have polynomial time solutions are called intractable.

# Introduction to P, NP, NP Hard & NP-Complete Problems

- **NP Class**

- An algorithm solves a problem in polynomial time if its worst -case time complexity belongs to  $O(p(n))$  where  $n$  is a size of a problem and  $p(n)$  is polynomial of the problem's input size  $n$ .
- Problems can be classified as tractable and intractable problems.
- Problems with polynomial time solutions are called tractable, problems which do not have polynomial time solutions are called intractable.



# Introduction to P, NP, NP Hard & NP-Complete Problems

- An NP algorithm consists of two stages:
  - Guessing Stage (Nondeterministic stage) : Given a problem instance, in this stage a simple string S is produced which can be thought of as a guess (candidate solution ) to the problem instance.
  - The following table displays the results of some input strings S generated at the nondeterministic stage for the problem instance graph given in figure 1 with the total distance d value that is claimed to be a tour not greater than d (i.e. 18) and passed to the function verify as an input.

| String S     | Output | Reasons                                             |
|--------------|--------|-----------------------------------------------------|
| [ A,B,C,D,A] | False  | Total weight of a tour( S string)is greater than 18 |
| [ A,D,B,C,A] | False  | S is not a correct tour                             |
| [ A,C,B,D,A] | True   | S is a tour with weight not exceeding 18            |

# Introduction to P, NP, NP Hard & NP-Complete Problems

- Verification Stage (Deterministic stage) : Input to this stage is the problem instance, the distance  $d$  and the string  $S$ . A deterministic algorithm takes these inputs and outputs yes if  $S$  is a correct guess to the problem instance and stops running. In case  $S$  is not a correct guess, then the deterministic algorithm outputs no or it may go to an infinite loop and does not halt.

Pseudocode

Boolean *verify*(weighted graph  $G$ ,  $d$ ,  $S$ )

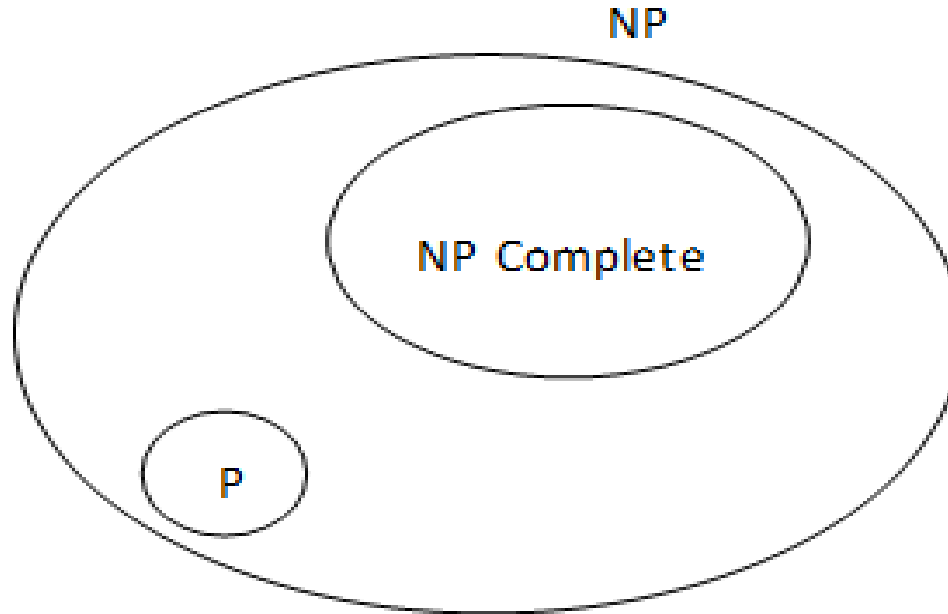
```
{
 if (S is a tour and the total weight of the edges in $S \leq d$)
 return True;
 else
 return False;
}
```

# Introduction to P, NP, NP Hard & NP-Complete Problems

- **NP Complete Problems**
  - NP complete problems are the most difficult problem, also called hardest problems in the subset of NP-class.
  - The common feature is that there is no polynomial time solution for any of these problems exists in the worst cases.
  - In other words, NP-Complete problems usually take super polynomial or exponential time or space in the worst cases.

# Introduction to P, NP, NP Hard & NP-Complete Problems

- NP Problems containing both P and NP-Complete Problems



# Introduction to P, NP, NP Hard & NP-Complete Problems

- NP-Complete problem can be reduced or mapped to it in polynomial time. To reduce one problem X to another problem Y,
- we need to do mapping such that a problem instance of X can be transformed to a problem instance of Y, solve Y and then do the mapping of the result back to the original problem

# CNF – Satisfiability Problem – A First NP Complete Problem

- Boolean Satisfiability Problem is also known as SAT is the problem of determining if there exists an interpretation that satisfies a given boolean formula.
- The variables of a given boolean formula can be consistently replaced by the values TRUE or FALSE that the formula evaluates to TRUE. Here, the formula is called satisfiable. On the other hand, if no such assignment exists, the function expressed by the formula is FALSE for all possible variable assignments and the formula is unsatisfiable.

# CNF – Satisfiability Problem – A First NP Complete Problem

- Let us recall that for a new problem to be NP-Complete problem , it must be in NPclass and then a known NP- Complete problem must be polynomials reduced to it.
- In this session we will not show any reduction example, because we are interested in the general idea.
- One might be wondering how the first NP-Complete problem was proven to be NP Complete and how the reduction was done.

# CNF – Satisfiability Problem – A First NP Complete Problem

- The satisfiability problem was the first NP-Complete problem ever found.
- The problem can be formulated as follows: Given a Boolean expression, find out whether the expression is satisfiable or not.
- Whether an assignment to the logical variables that gives a value 1(True) A logical variable, also called a Boolean variable is a variable having only one of the two values: 1 (True) or False (0). A literal is a logical variable or negation of a logical variable. A clause combines literals through logical or( $\vee$ ) operator(s). A conjunctive normal form (CNF) combines several clauses through logical and ( $\wedge$ ) operator(s)



# Important Topics

- Difference between Optimization vs Decision Problems
- P, NP, and NP-Complete Problems
- The CNF Satisfiability Problem

# Summary

- In this session we have seen that :-
- Relationship between P, NP, NP-Complete
- Three classes of problems in terms of its time complexities in the worst cases: P, NP, NP-Complete.
- Circuit Satisfiability Decision Problem asks for a given logical expression in CNF, Whether some combination of True and False values to the logical variables makes the output of the expression
- Differences between tractable and intractable problems, optimization and decision problems and deterministic and nondeterministic algorithms

*Thank  
You*



# **Design and Analysis of Algorithms**

## **Block-4 Unit-2**

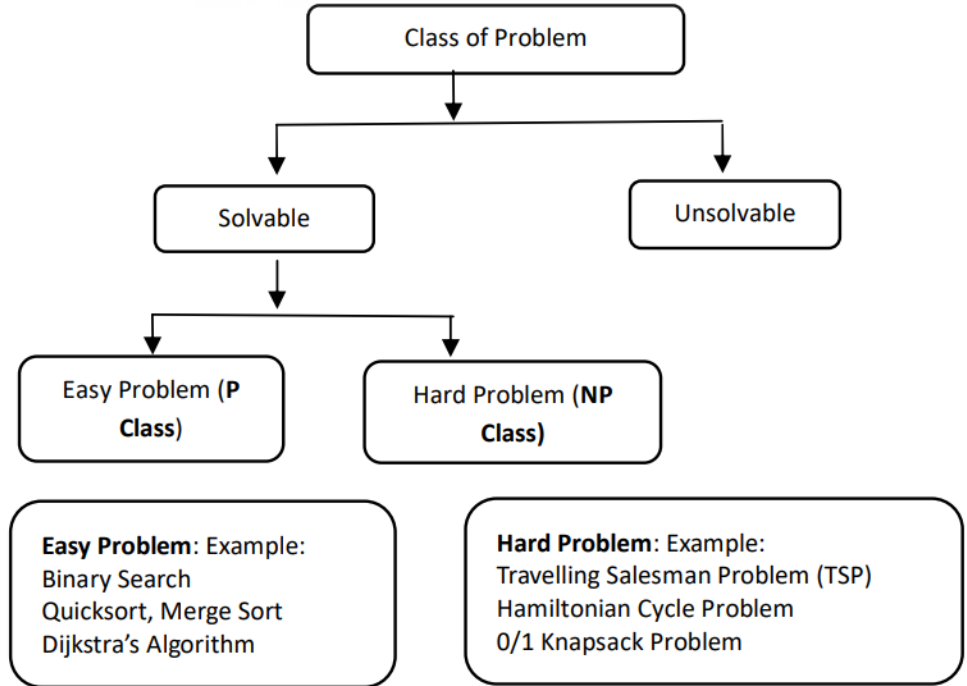
### **NP-Completeness And NP-Hard Problems**

# Topics to be Covered

- Introduction
- P Vs NP-Class of Problems
- Polynomial time reduction
- NP-Hard and NP-Complete problem
- Some well-known NP-Complete Problems-definitions
- Techniques (Steps) for proving NP-Completeness
- Proving NP-completeness (Decision problems)
- Important Topics
- Summary

# Introduction

- A class of problems can be divided into two parts: solvable and unsolvable problems. Solvable problems are those for which an algorithm exist, and unsolvable problems are those for which algorithm does not exist such as halting problem of turing machine etc.



# P Vs NP-Class of Problems

- **P-Class (Polynomial class):**
  - If there exist a polynomial time algorithm for L then
  - problem L is said to be in class P (Polynomial Class), that is in worst case L can be solved in where n is the input size of the problem and k is positive constant.  $(n^k)$

$$P = \left\{ L \mid \begin{array}{l} L \text{ can be decided (accepted) by deterministic Turing machine} \\ (\text{algorithm}) \in \text{polynomial time} \end{array} \right\}$$

# P Vs NP-Class of Problems

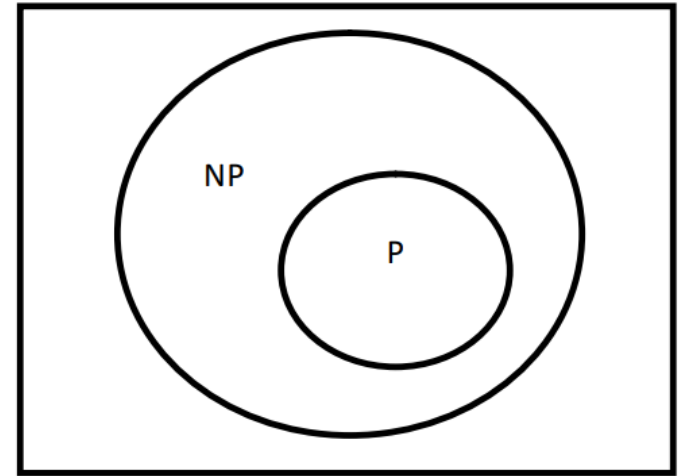
- **NP-Class (Nondeterministic Polynomial time solvable):**
  - NP is the set of decision problems (with 'yes' or 'no' answer) that can be solved by a Nondeterministic algorithm (or Turing Machine) in Polynomial time.

$$NP = \left\{ L \mid \begin{array}{l} L \text{ can be decided (accepted) by Nondeterministic Turing machine} \\ (\text{algorithm}) \in \text{polynomial time} \end{array} \right\}$$



# P Vs NP-Class of Problems

- Here is an example of an NDA:
- Algorithm: (Nondeterministic Linear search)
- /\* Input: A linear list A with n elements and a searching element  $x$ .
- Output: Finds the location LOC of  $x$  in the array A (by returning an index) or return LOC=0 to indicate  $x$  is not present in A. \*/
- *NDLSearch*(A, n, x) {
- 1. [Initialize]: Set  $j=1$  and LOC=0.
- 2.  $j = \text{Choice}()$ ; // Nondeterministic Statement
- 3. *if*( $x = A[j]$ ) {
- 5. LOC= $j$
- 6. *printf*}
- 8.
- *if*(LOC = 0)
- 9. *printf* }



Relationship between P  
and NP class of problem

# Polynomial Time Reduction

- An any new problem requires a new algorithm, But often we can solve a problem  $X$  using a known algorithm for a related problem  $Y$ .
- A given instance  $x$  of  $X$  is translated to a suitable instance  $y$  of  $Y$ , so that we can use the available algorithm for  $Y$ . Eventually the result of this computation on  $y$  is translated back, so that we get the desired result for  $x$ .

# Polynomial Time Reduction

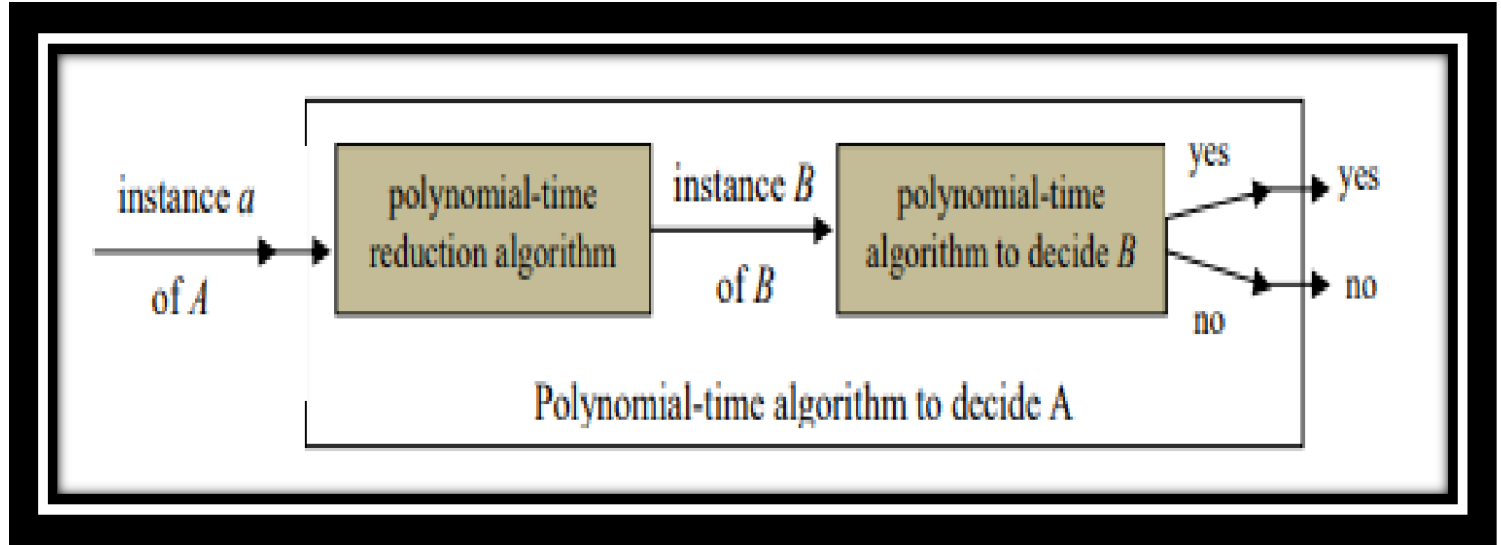
- **Reductions-Formal definition:**
  - Reduction is a general technique for showing similarity between problems. To show the similarity between problems we need one base problem. A procedure which is used to show the relationship (or similarity) between problems is called Reduction step, and symbolically can be written as

$$A \leq_p B$$

# Polynomial Time Reduction

- The meaning of the above statement is “problem A is polynomial time reducible to problem B” and if there exist a polynomial time algorithm for problem A then problem B can also have polynomial time algorithm.
- Here problem A is taken as a base problem

# Polynomial Time Reduction



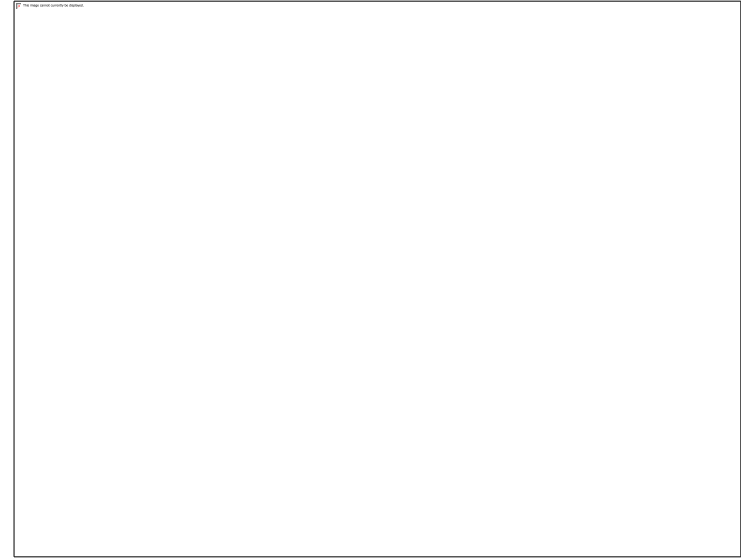
Reduction step of problem A to problem B

# NP-Hard and NP-Complete Problems

- NP-Hard or NP-Complete problems are stated in terms of language recognition problems. This is because the theory of NP-Completeness grew out of automata and formal language theory.
- NP-Complete problems are the “hardest” problems to solve among all NP problems, in that if one of them can be solved in polynomial time then every problem in NP can be solved in polynomial time.

# NP-Hard and NP-Complete Problems

- *Easy*  $\rightarrow P$
- *Medium*  $\rightarrow NP$
- *Hard*  $\rightarrow NP - Complete$
- *Hardest*  $\rightarrow NP - Hard$
- The following figure 6 shows a relationship between P, NP, NP-C and NP-Hard problems:



Source : <https://www.baeldung.com/cs/p-np-np-complete-np-hard>

# NP-Hard and NP-Complete Problems

- **NP-Hard Problem**

- A problem  $L$  is said to be NP-Hard problem if there is a polynomial time reduction from
- already known NP-Hard problem  $L_0$  to the given problem  $L$ .

- $E' = \{(i, j) : i, j \in V \text{ and } i \neq j\}$



# Some Well-Known Np-complete Problems Definitions

- **SAT Problem**
  - **Input:** Given a CNF formula  $f$  having  $m$  clauses  $C_1, C_2, \dots, C_m$  in  $n$  variables  $x_1, x_2, \dots, x_n$ . There is no restriction on number of variables in each clause. The problem is to find an assignment (value) to a set of variables  $x_1, x_2, \dots, x_n$  which satisfy all the  $m$  clauses simultaneously.
  - $SAT = \{f: f \text{ is a given Boolean formula in CNF, Is this formula } f \text{ satisfiable ?}\}$

# Some Well-Known Np-complete Problems Definitions

- **NP-Complete Problem**

- A decision problem is said to be NP-Complete if the following two conditions are satisfied

$$L \in NP$$

- L is NP Hard (that is every problem (say  $L'$ ) in NP is polynomial-time reducible to L).

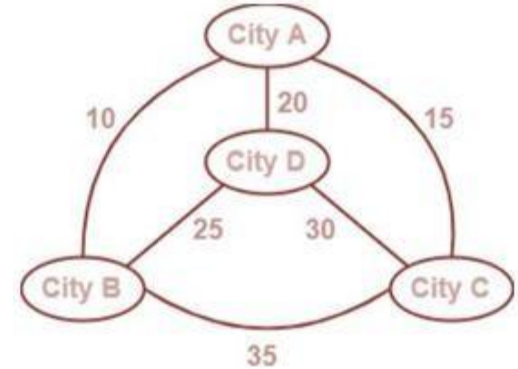
# Definitions of Some Well-Known NP- Complete Problems

- Knapsack problem:
  - A decision version of 0/1 knapsack problem is NP- complete. A list  $\{i_1, i_2, \dots, i_n\}$ , a knapsack with capacity  $M$  and a desired value  $V$ . Each object  $i_i$  has a weight  $w_i$  and value  $v_i$ . Can a subset of items  $X \subseteq \{i_1, i_2, \dots, i_n\}$  be picked whose total weight is at most  $M$  and at total value is at least  $V$ ? that satisfy the following constraints:

$$\text{Maximize (the value)} \sum_{i=1}^n v_i x_i \text{ such that } \sum_{i=1}^n w_i x_i \leq M \text{ and } x_i \in \{0,1\}$$

# Definitions of Some Well-Known NP- Complete Problems

- **Traveling Salesman Problem (TSP)**
- Given a set of cities and distance between every pair of cities, the problem is to find the shortest possible route that visits every city exactly once and returns to the starting point. There is a integer cost  $C(i, j)$  to travel from city  $i$  to city  $j$  and the salesman wishes to make the tour whose total cost is minimum, where the total cost is the sum of individual costs along the edges of the tour.



$G = (V, E)$  is a complete graph,  $C$   
is the function

$$TSP = \langle G, C, k \rangle_1$$

$$V \times V \rightarrow \mathbb{Z}, k \in \mathbb{Z} \wedge G \text{ has a}$$

# Definitions of Some Well-Known NP- Complete Problems

- **Sum-of-Subset Problem**

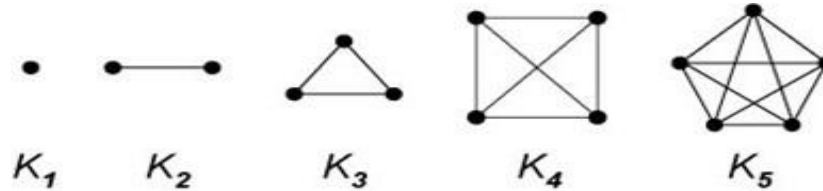
- Given set of positive integer  $S = \{x_1, x_2, \dots, x_n\}$  and
- a target sum  $K$ , the decision problem asks for a subset  $S'$  of  $S$  (i. e.  $S' \subseteq S$ ) having a sum equal to  $K$ .

$$SUBSET-SUM = \{\langle S, t \rangle \mid S = \{x_1, \dots, x_k\}, \text{ and for some } \{y_1, \dots, y_l\} \subseteq \{x_1, \dots, x_k\}, \text{ we have } \sum y_i = t\}.$$

# Definitions of some well-known NP-Complete problems

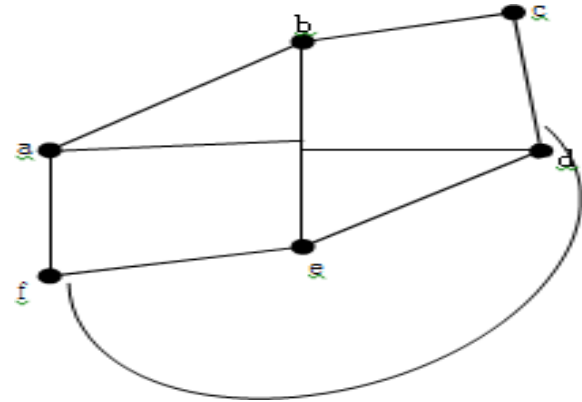
- **CLIQUE Problem**

- CLIQUE is a complete Subgraph problem. A clique in an Undirected graph  $G = (V, E)$  is a subset  $V' \subseteq V$ , each pair of which is connected by an edge in  $E$  (a complete Subgraph of  $G$ ).
- $\text{CLIQUE} = \{\langle G, k \rangle : G \text{ is a graph containing a clique of size } k\}$



# Definitions of Some Well-Known NP- Complete Problems

- **Vertex Cover Problem (VCP):**
  - A vertex cover of an undirected graph  $G = (V, E)$  is a subset of the vertices  $V' \subseteq V$  such that for any edge  $(U, V) \in E$ , then either  $u \in V'$  or  $v \in V'$  or both.



# Definitions of Some Well-Known NP- Complete Problems

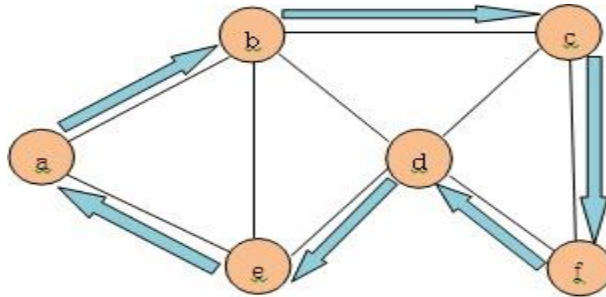
- **Vertex Cover as a Decision Problem**
  - The problem vertex cover, stated as a decision problem, is to determine whether a given graph  $G(V, E)$  with  $|V|=n$ , has a vertex cover of size  $k$ , where  $k \leq n$ . VERTEX-COVER =  $\{\langle G, k \rangle: \text{graph } G \text{ has a vertex cover of size } k\}$



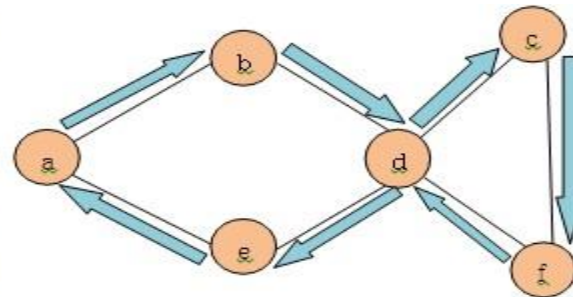
# Definitions of Some Well-Known NP- Complete Problems

- **Hamiltonian Cycle problem**

- A Hamiltonian Cycle of an undirected graph  $G(V, E)$  is a simple cycle that passes through all the vertices of the graph  $G$  exactly once. For example, the following graph as shown in figure, contains a cycle  $a - b - c - f - d - e - a$  that visits each vertex exactly once.



Hamiltonian Graph



Non Hamiltonian

# Definitions of Some Well-Known NP- Complete Problems

- **Graph Coloring Problem:**

- The natural graph coloring optimization problem is to color a graph with the fewest number of colors. it is based on decision problem and provide the input is a pair  $(G, k)$  and then
- The search problem is to find a  $k$ -coloring of the graph  $G$  if one exists.
- The decision problem is to determine whether or not  $G$  has a  $k$  coloring. Clearly solving the optimization problem solves the search problem which in turn solves the decision problem.

# Important Topics

- P Vs NP-Class of Problems
- Polynomial time reduction
- NP-Hard and NP-Complete problem
- The CNF Satisfiability Problem
- Techniques (Steps) for proving NP- Completeness
- NP-completeness (Decision problems)

# Summary

- In this session we have seen that :-
- A class of problems can be divided into two parts: solvable and unsolvable problems. Solvable problems are those for which an algorithm exist, and unsolvable problems are those for which algorithm does not exist such as halting problem of turing machine etc.
- Polynomial time algorithm for  $L$  then problem  $L$  is said to be in class  $P$  (Polynomial Class), that is in worst case  $L$  can be solved in  $O(n^k)$  where  $n$  is the input size of the problem and  $k$  is positive constant.
- A problem  $L$  is said to be NP-Hard problem if there is a polynomial time reduction from already known NP-Hard problem  $L'$  to the given problem  $L$ .

Thank  
You



# **Design and Analysis of Algorithms**

## **Block-4 Unit-13**

### **Handling Intractability**

# Topics to be Covered

- Introduction
- Intelligent Exhaustive Search
- Approximation Algorithms Basics
- Important Topics
- Summary

# Introduction

- Problem solving techniques such as backtracking and branch and bound techniques perform better in comparison to exhaustive search.
- But unlike exhaustive search, these techniques construct the solutions step by step and performs evaluation of the partial solution.
- The focus of the unit will be to discuss few techniques such as backtracking and branch and bound techniques and approximation algorithms to handle intractable problems.



# Backtracking and Branch and Bound Techniques

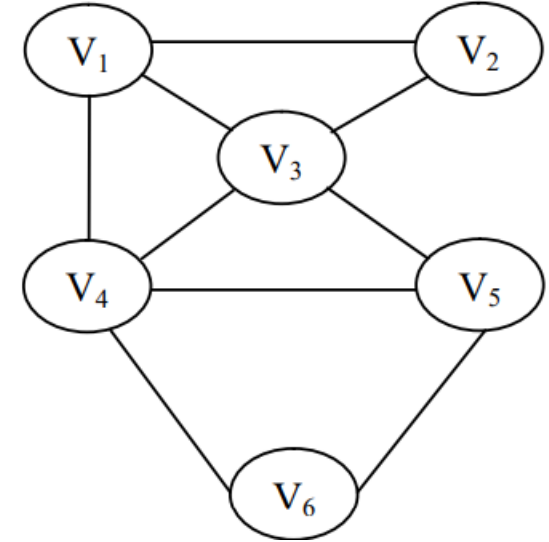
- Backtracking
  - Backtracking is a technique for design of algorithms. It is applied to solve problems in which components are selected in sequence from the specified set so that it satisfies some criteria or objective.
  - Backtracking procedure includes depth first search of a state space tree, verifying whether a node is leading to any solution(called promising node)or but dead ends (called non promising node), does backtracking to the parent of the node if a node is not promising and continuing with the search process on the next child.

# Backtracking and Branch and Bound Techniques

- Backtracking technique to solve two problems
  - Hamiltonian Circuit problem
  - Subset Sum problem

# Backtracking and Branch and Bound Techniques

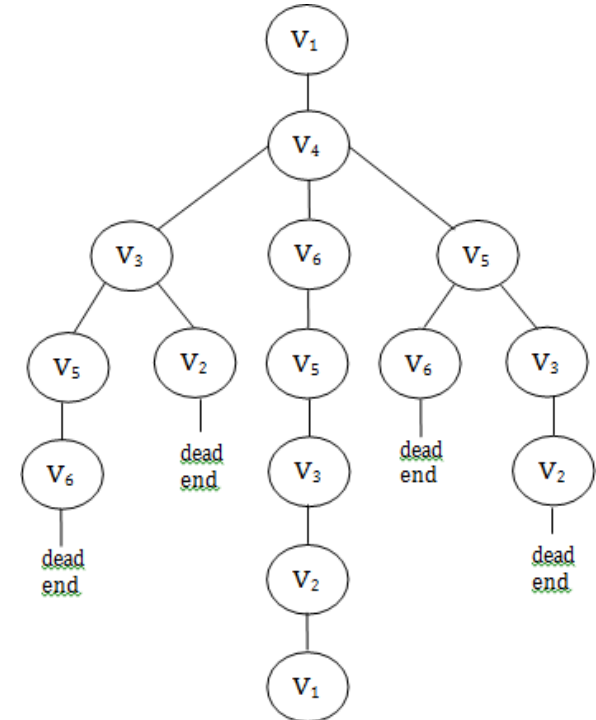
- **Hamiltonian circuit problem**
  - Suppose  $G = (V, E)$  is a connected graph with  $n$  vertices, Hamiltonian circuit problem determines a cycle that visits every vertex of a graph only once except the starting
  - vertex.  $V_1$  is a starting vertex of a cycle where  $V_1 \in G$  and  $V_1, V_2, \dots, V_{n+1}$  are distinct vertices in the cycle except  $V_1$  and  $V_{n+1}$  vertices which are equal.



A graph for finding Hamiltonian cycle

# Backtracking and Branch and Bound Techniques

- **Design of a state space tree**
  - $V_1$  as the starting vertex in a state space tree of a graph given at figure Among the three options available to select a node from  $V_1$ , we pick up  $V_3$
  - From  $V_3$  it can go to  $V_5$  or  $V_2$  or  $V_6$
  - If we follow the order of  $V_6, V_5, V_2$  and  $V_1$ , we get a correct Hamiltonian cycle.



State Space Tree representing Hamiltonian circuit of a graph

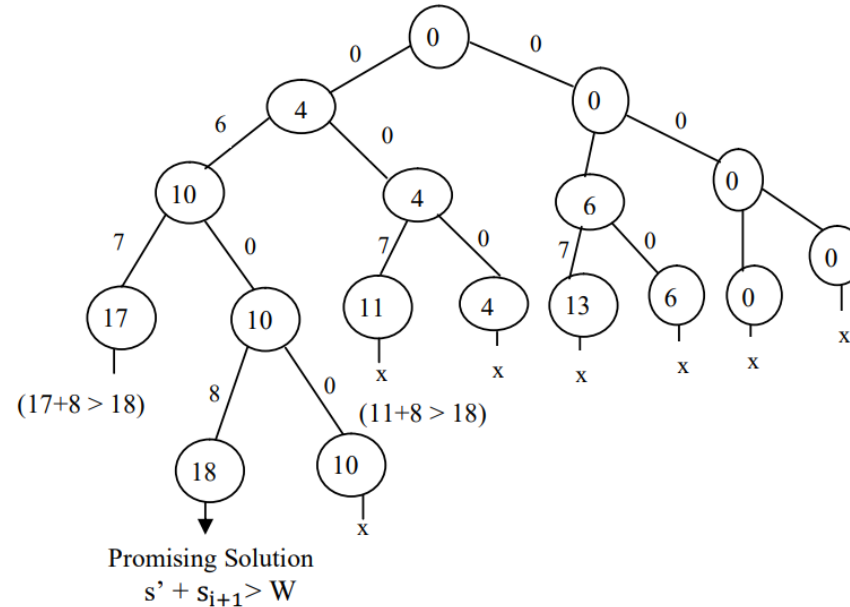
# Backtracking and Branch and Bound Techniques

- **Subset sum Problem**

- Given a positive integer  $W$  and a set  $S$  of  $n$  positive integer values i.e.,  $S = \{s_1, s_2, \dots, s_n\}$ , the main objective of the Subset Sum problem is to search for all combinations of subsets of integers whose sum is equal to  $W$ .
- For example let us take  $S = \{1, 4, 6, 9\}$  and  $W = 10$ . there are two solution subsets :  $\{1, 9\}$  and  $\{4, 6\}$ . In some cases, problem instances may not have any solution subset.
- We will assume that elements in the set are in the sorted order, i.e.,  $s_1 \leq s_2 \leq \dots \leq s_n$ .

# Backtracking and Branch and Bound Techniques

- Design of state space tree for subset sum problem:  $S = \{4, 6, 7, 8\}$  and  $W = 18$



# Backtracking and Branch and Bound Techniques

- **Branch and Bound**

- In branch and bound technique, search path in state space tree at a particular node is stopped if any of the following reasons occurs:
  - Constraint violation
  - The value of a bound of a node is inferior to the value of the v best solution achieved so far.

# Backtracking and Branch and Bound Techniques

- As in backtracking, the left branch of the tree includes the next object while the right branch excludes it. A node is represented by the three values:
  - $w$  – total weight of items selected at a node
  - $p$  – total profit
  - bound – total profits of any subset of items
- One of the simplest way to calculate the bound is given below:

$$\text{bound} = p + (W-w)(p_{i+1}/w_{i+1})$$



# Approximation Algorithms Basics

- A radically approach to deal with hard combinatorial problems called approximation algorithms which provide a solution to the hard combinatorial problem, reasonably close to optimal solution but in polynomial time.
- In real life situations usually work with inaccurate data. In such cases, getting near optimal solution is accepted and good enough.

# Approximation Algorithms Basics

- Its require to find a bound that provides a measure how close a proposed solution is to optimal solution.
- For example, consider a TSP(Traveling Salesperson Optimization Problem) problem which is NP- Hard .

# Approximation Algorithms Basics

- There is an algorithm whose solution has the following guarantee: approx. value is less than twice optimal value (i.e.  $< 2 \times \text{opt value}$ ) where, opt value is the optimal solution for the problem and approx. value is the approximate solution that the algorithm outputs.

# Approximation Algorithms Basics

- **Approximation Ratio**

- The term approximation ratio which can be defined as a ratio between the results obtained by the approximation algorithm and the optimal results of the algorithm. Consider a minimization optimization problem  $P$  in which the main task is to minimize the given objective function.
- For example, vertex cover problem, graph coloring problem, etc. Are combinatorial minimization optimization problem.

# Important Topics

- Intelligent Exhaustive Search and explain their types
- Approximation Algorithms Basics

# Summary

- In this session we have seen that :-
- Problem solving techniques such as backtracking and branch and bound techniques perform better in comparison to exhaustive search. But unlike exhaustive search, these techniques construct the solutions step by step and perform evaluation of the partial solution.
- A radically approach to deal with hard combinatorial problems called approximation algorithms which provide a solution to the hard combinatorial problem, reasonably close to optimal solution but in polynomial time.

*Thank  
You*